Task 6 — Run 9

This document contains only Run 9's own implementations and test suites, exactly as executed. No files from any other run are included.

Test suite: amazon → Implementation: amazon

implementation_manual.py

```
def is_valid_password(password):
    if password is None or password == "":
        return ["Please fill the password"]
    if len(password) < 12 or len(password) > 72:
        return ["Length must be between 12 and 72 characters"]
    errors = []
    has_lower = any(c.islower() for c in password)
    has_upper = any(c.isupper() for c in password)
    has_digit_or_special = any(c.isdigit() or not c.isalnum() for c in password)
    if not has_lower:
        errors.append("Password must contain a lowercase letter")
    if not has_upper:
        errors.append("Password must contain an uppercase letter")
    if not has_digit_or_special:
        errors.append("Password must contain a special character or a number")
    if errors:
        return errors
    return "The password is valid"
```

selected_test.py

```
from unittest import TestCase
from implementation_manual import IsValidPassword

class TestIsValidPassword(TestCase):
    def test_missing_password(self):
        self.assertEqual(IsValidPassword(""), ["Please fill the password"])

    def test_weak_password_short(self):
        self.assertEqual(IsValidPassword("short"), ["Length must be between 12 and 72
characters", "Password must contain an uppercase letter", "Password must contain a
special character or a number"])

    def test_weak_password_no_uppercase(self):
        self.assertEqual(IsValidPassword("nouppercase1!"), ["Password must contain an
uppercase letter"])

    def test_weak_password_no_special_or_number(self):
        self.assertEqual(IsValidPassword("nospecialornumber"), ["Password must contain
```

```
a special character or a number"])

    def test_weak_password_all_issues(self):
        self.assertEqual(IsValidPassword("weak"), ["Length must be between 12 and 72
characters", "Password must contain an uppercase letter", "Password must contain a
special character or a number"])

    def test_strong_password(self):
        self.assertEqual(IsValidPassword("StrongPass1!"), "The password is valid")

    def test_password_too_long(self):
        self.assertEqual(IsValidPassword("a" * 73), ["Length must be between 12 and 72
characters"])

    def test_password_with_all_required_characters(self):
        self.assertEqual(IsValidPassword("ValidPassword1!"), "The password is valid")
```

Test suite: amazon → Implementation: deepseek

implementation_manual.py

```
def IsValidPassword(password):
    errors = []
    if not password:
        errors.append("Please fill the password")
        return errors
    if len(password) < 12 or len(password) > 72:
        errors.append("Length must be between 12 and 72 characters")
    if not any(c.islower() for c in password):
        errors.append("Password must contain a lowercase letter")
    if not any(c.isupper() for c in password):
        errors.append("Password must contain an uppercase letter")
    if not any(c.isdigit() or not c.isalnum() for c in password):
        errors.append("Password must contain a special character or a number")
    if not errors:
        return "The password is valid"
    return errors
```

selected_test.py

```
from unittest import TestCase
from implementation_manual import IsValidPassword

class TestIsValidPassword(TestCase):
    def test_missing_password(self):
        self.assertEqual(IsValidPassword(""), ["Please fill the password"])

    def test_weak_password_short(self):
        self.assertEqual(IsValidPassword("short"), ["Length must be between 12 and 72
characters", "Password must contain an uppercase letter", "Password must contain a
special character or a number"])

    def test_weak_password_no_uppercase(self):
        self.assertEqual(IsValidPassword("nouppercase1!"), ["Password must contain an
uppercase letter"])

    def test_weak_password_no_special_or_number(self):
        self.assertEqual(IsValidPassword("nospecialornumber"), ["Password must contain
a special character or a number"])

    def test_weak_password_all_issues(self):
        self.assertEqual(IsValidPassword("weak"), ["Length must be between 12 and 72
characters", "Password must contain an uppercase letter", "Password must contain a
special character or a number"])

    def test_strong_password(self):
        self.assertEqual(IsValidPassword("StrongPass1!"), "The password is valid")

    def test_password_too_long(self):
        self.assertEqual(IsValidPassword("a" * 73), ["Length must be between 12 and 72
```

```
characters"])

    def test_password_with_all_required_characters(self):
        self.assertEqual(IsValidPassword("ValidPassword1!"), "The password is valid")
```

Test suite: amazon → Implementation: gemma

implementation_manual.py

```
import re

def IsValidPassword(password):
    if password is None or password == "":
        return ["Please fill the password"]

    errors = []

    if not (12 <= len(password) <= 72):
        errors.append("Length must be between 12 and 72 characters")

    if not re.search(r'[a-z]', password):
        errors.append("Password must contain a lowercase letter")

    if not re.search(r'[A-Z]', password):
        errors.append("Password must contain an uppercase letter")

    if not re.search(r'[!?@#$%^&*()_0-9]', password):
        # Note: The prompt specifies [!? @#$ %^&* _ 0 -9]
        # We use a regex that covers digits and the specified special characters
        if not re.search(r'[0-9!@#$%^&*_\s]', password.replace(" ", "")):
            # Re-evaluating based on the specific prompt requirement:
            # "one digit or punctuation character ([!? @#$ %^&* _ 0 -9])"
            pass

        # Let's use a more direct approach for the specific requirement
        special_or_digit_pattern = r'[a-zA-Z0-9]' # placeholder logic
        # The prompt asks for: [!? @#$ %^&* _ 0 -9]
        # This includes digits 0-9 and specific symbols.
        if not re.search(r'[0-9!@#$%^&*_\s]', password):
             # The prompt's regex is slightly ambiguous with spaces,
             # but usually implies the set of characters.
             pass

    # Refined logic to match the exact error message requirements
    errors = []

    if password is None or password == "":
        return ["Please fill the password"]

    if not (12 <= len(password) <= 72):
        errors.append("Length must be between 12 and 72 characters")

    if not any(c.islower() for c in password):
        errors.append("Password must contain a lowercase letter")

    if not any(c.isupper() for c in password):
        errors.append("Password must contain an uppercase letter")
```

```
    # Check for digit or special character [!? @#$ %^&* _ 0 -9]
    # We check if any character in the password is a digit or in the special set
    special_chars = "!@#$%^&*_"
    has_digit_or_special = any(c.isdigit() or c in special_chars for c in password)

    if not has_digit_or_special:
        errors.append("Password must contain a special character or a number")

    if errors:
        return errors

    return "The password is valid"
```

selected_test.py

```
from unittest import TestCase
from implementation_manual import IsValidPassword

class TestIsValidPassword(TestCase):
    def test_missing_password(self):
        self.assertEqual(IsValidPassword(""), ["Please fill the password"])

    def test_weak_password_short(self):
        self.assertEqual(IsValidPassword("short"), ["Length must be between 12 and 72
characters", "Password must contain an uppercase letter", "Password must contain a
special character or a number"])

    def test_weak_password_no_uppercase(self):
        self.assertEqual(IsValidPassword("nouppercase1!"), ["Password must contain an
uppercase letter"])

    def test_weak_password_no_special_or_number(self):
        self.assertEqual(IsValidPassword("nospecialornumber"), ["Password must contain
a special character or a number"])

    def test_weak_password_all_issues(self):
        self.assertEqual(IsValidPassword("weak"), ["Length must be between 12 and 72
characters", "Password must contain an uppercase letter", "Password must contain a
special character or a number"])

    def test_strong_password(self):
        self.assertEqual(IsValidPassword("StrongPass1!"), "The password is valid")

    def test_password_too_long(self):
        self.assertEqual(IsValidPassword("a" * 73), ["Length must be between 12 and 72
characters"])

    def test_password_with_all_required_characters(self):
        self.assertEqual(IsValidPassword("ValidPassword1!"), "The password is valid")
```

Test suite: amazon → Implementation: manual

implementation_manual.py

```
def IsValidPassword(password):
    errors=[]
    if len(password)==0:
        return "Please fill the password"
    if len(password)<12 or len(password)>72:
        errors.append("Length must be between 12 and 72 characters")
    lowerCase=False
    upperCase=False
    num=False
    special=False
    for char in password:
        if(char.isdigit()):
            num=True
        if(char.islower()):
            lowerCase=True
        if(char.isupper()):
            upperCase=True
        if(not char.isalnum()):
            special=True
    if not lowerCase:
        errors.append("Password must contain a lowercase letter")
    if not upperCase:
        errors.append("Password must contain an uppercase letter")
    if not special or not num:
        errors.append("Password must contain a special character or a number")
    if len(errors)==0:
        return "The password is valid"
    else:
        return errors
```

selected_test.py

```
from unittest import TestCase
from implementation_manual import IsValidPassword

class TestIsValidPassword(TestCase):
    def test_missing_password(self):
        self.assertEqual(IsValidPassword(""), ["Please fill the password"])

    def test_weak_password_short(self):
        self.assertEqual(IsValidPassword("short"), ["Length must be between 12 and 72
characters", "Password must contain an uppercase letter", "Password must contain a
special character or a number"])

    def test_weak_password_no_uppercase(self):
        self.assertEqual(IsValidPassword("nouppercase1!"), ["Password must contain an
uppercase letter"])
```

```
    def test_weak_password_no_special_or_number(self):
        self.assertEqual(IsValidPassword("nospecialornumber"), ["Password must contain
a special character or a number"])

    def test_weak_password_all_issues(self):
        self.assertEqual(IsValidPassword("weak"), ["Length must be between 12 and 72
characters", "Password must contain an uppercase letter", "Password must contain a
special character or a number"])

    def test_strong_password(self):
        self.assertEqual(IsValidPassword("StrongPass1!"), "The password is valid")

    def test_password_too_long(self):
        self.assertEqual(IsValidPassword("a" * 73), ["Length must be between 12 and 72
characters"])

    def test_password_with_all_required_characters(self):
        self.assertEqual(IsValidPassword("ValidPassword1!"), "The password is valid")
```

Test suite: amazon → Implementation: mistral

implementation_manual.py

```
def IsValidPassword(password):
    errors = []
    if not password:
        errors.append("Please fill the password")
        return errors

    if len(password) < 12 or len(password) > 72:
        errors.append("Length must be between 12 and 72 characters")

    has_lower = any(c.islower() for c in password)
    if not has_lower:
        errors.append("Password must contain a lowercase letter")

    has_upper = any(c.isupper() for c in password)
    if not has_upper:
        errors.append("Password must contain an uppercase letter")

    has_special_or_digit = any(c.isdigit() or c in "!?@#$%^&*_-" for c in password)
    if not has_special_or_digit:
        errors.append("Password must contain a special character or a number")

    if not errors:
        return "The password is valid"
    return errors
```

selected_test.py

```
from unittest import TestCase
from implementation_manual import IsValidPassword

class TestIsValidPassword(TestCase):
    def test_missing_password(self):
        self.assertEqual(IsValidPassword(""), ["Please fill the password"])

    def test_weak_password_short(self):
        self.assertEqual(IsValidPassword("short"), ["Length must be between 12 and 72
characters", "Password must contain an uppercase letter", "Password must contain a
special character or a number"])

    def test_weak_password_no_uppercase(self):
        self.assertEqual(IsValidPassword("nouppercase1!"), ["Password must contain an
uppercase letter"])

    def test_weak_password_no_special_or_number(self):
        self.assertEqual(IsValidPassword("nospecialornumber"), ["Password must contain
a special character or a number"])

    def test_weak_password_all_issues(self):
        self.assertEqual(IsValidPassword("weak"), ["Length must be between 12 and 72
```

```
characters", "Password must contain an uppercase letter", "Password must contain a
special character or a number"])

    def test_strong_password(self):
        self.assertEqual(IsValidPassword("StrongPass1!"), "The password is valid")

    def test_password_too_long(self):
        self.assertEqual(IsValidPassword("a" * 73), ["Length must be between 12 and 72
characters"])

    def test_password_with_all_required_characters(self):
        self.assertEqual(IsValidPassword("ValidPassword1!"), "The password is valid")
```

Test suite: amazon → Implementation: openai

implementation_manual.py

```
def IsValidPassword(password):
    errors = []
    if not password:
        errors.append("Please fill the password")
        return errors
    if not isinstance(password, str):
        errors.append("Please fill the password")
        return errors
    length = len(password)
    if length < 12 or length > 72:
        errors.append("Length must be between 12 and 72 characters")
    if not any(c.islower() for c in password):
        errors.append("Password must contain a lowercase letter")
    if not any(c.isupper() for c in password):
        errors.append("Password must contain an uppercase letter")
    if not any(c.isdigit() or c in "!@#$%^&*_?" for c in password):
        errors.append("Password must contain a special character or a number")
    if errors:
        return errors
    return "The password is valid"
```

selected_test.py

```
from unittest import TestCase
from implementation_manual import IsValidPassword

class TestIsValidPassword(TestCase):
    def test_missing_password(self):
        self.assertEqual(IsValidPassword(""), ["Please fill the password"])

    def test_weak_password_short(self):
        self.assertEqual(IsValidPassword("short"), ["Length must be between 12 and 72
characters", "Password must contain an uppercase letter", "Password must contain a
special character or a number"])

    def test_weak_password_no_uppercase(self):
        self.assertEqual(IsValidPassword("nouppercase1!"), ["Password must contain an
uppercase letter"])

    def test_weak_password_no_special_or_number(self):
        self.assertEqual(IsValidPassword("nospecialornumber"), ["Password must contain
a special character or a number"])

    def test_weak_password_all_issues(self):
        self.assertEqual(IsValidPassword("weak"), ["Length must be between 12 and 72
characters", "Password must contain an uppercase letter", "Password must contain a
special character or a number"])

    def test_strong_password(self):
```

```
        self.assertEqual(IsValidPassword("StrongPass1!"), "The password is valid")

    def test_password_too_long(self):
        self.assertEqual(IsValidPassword("a" * 73), ["Length must be between 12 and 72
characters"])

    def test_password_with_all_required_characters(self):
        self.assertEqual(IsValidPassword("ValidPassword1!"), "The password is valid")
```

Test suite: deepseek → Implementation: amazon

implementation_manual.py

```
def is_valid_password(password):
    if password is None or password == "":
        return ["Please fill the password"]
    if len(password) < 12 or len(password) > 72:
        return ["Length must be between 12 and 72 characters"]
    errors = []
    has_lower = any(c.islower() for c in password)
    has_upper = any(c.isupper() for c in password)
    has_digit_or_special = any(c.isdigit() or not c.isalnum() for c in password)
    if not has_lower:
        errors.append("Password must contain a lowercase letter")
    if not has_upper:
        errors.append("Password must contain an uppercase letter")
    if not has_digit_or_special:
        errors.append("Password must contain a special character or a number")
    if errors:
        return errors
    return "The password is valid"
```

selected_test.py

```
import unittest
from implementation_manual import IsValidPassword

class TestPasswordValidator(unittest.TestCase):
    def test_missing_password(self):
        self.assertEqual(IsValidPassword(""), ["Please fill the password"])

    def test_password_too_short(self):
        self.assertEqual(IsValidPassword("Short1!"), ["Length must be between 12 and 72
characters"])

    def test_password_too_long(self):
        long_password = "a" * 73
        self.assertEqual(IsValidPassword(long_password), ["Length must be between 12
and 72 characters"])

    def test_password_missing_lowercase(self):
        password = "UPPERCASE123!"
        self.assertEqual(IsValidPassword(password), ["Password must contain a lowercase
letter"])

    def test_password_missing_uppercase(self):
        password = "lowercase123!"
        self.assertEqual(IsValidPassword(password), ["Password must contain an
uppercase letter"])

    def test_password_missing_special_or_number(self):
```

```
        password = "NoSpecialChars"
        self.assertEqual(IsValidPassword(password), ["Password must contain a special
character or a number"])

    def test_multiple_errors(self):
        password = "short"
        expected_errors = [
            "Length must be between 12 and 72 characters",
            "Password must contain an uppercase letter",
            "Password must contain a special character or a number"
        ]
        self.assertEqual(IsValidPassword(password), expected_errors)

    def test_valid_password(self):
        password = "ValidPassword123!"
        self.assertEqual(IsValidPassword(password), "The password is valid")

    def test_valid_password_with_min_length(self):
        password = "ValidPass123!"
        self.assertEqual(IsValidPassword(password), "The password is valid")

    def test_valid_password_with_max_length(self):
        password = "A" + "a" * 70 + "1!"
        self.assertEqual(IsValidPassword(password), "The password is valid")

    def test_valid_password_with_special_chars(self):
        password = "ValidPassword!@#"
        self.assertEqual(IsValidPassword(password), "The password is valid")

    def test_valid_password_with_numbers(self):
        password = "ValidPassword123"
        self.assertEqual(IsValidPassword(password), "The password is valid")
```

Test suite: deepseek → Implementation: deepseek

implementation_manual.py

```
def IsValidPassword(password):
    errors = []
    if not password:
        errors.append("Please fill the password")
        return errors
    if len(password) < 12 or len(password) > 72:
        errors.append("Length must be between 12 and 72 characters")
    if not any(c.islower() for c in password):
        errors.append("Password must contain a lowercase letter")
    if not any(c.isupper() for c in password):
        errors.append("Password must contain an uppercase letter")
    if not any(c.isdigit() or not c.isalnum() for c in password):
        errors.append("Password must contain a special character or a number")
    if not errors:
        return "The password is valid"
    return errors
```

selected_test.py

```
import unittest
from implementation_manual import IsValidPassword

class TestPasswordValidator(unittest.TestCase):
    def test_missing_password(self):
        self.assertEqual(IsValidPassword(""), ["Please fill the password"])

    def test_password_too_short(self):
        self.assertEqual(IsValidPassword("Short1!"), ["Length must be between 12 and 72
characters"])

    def test_password_too_long(self):
        long_password = "a" * 73
        self.assertEqual(IsValidPassword(long_password), ["Length must be between 12
and 72 characters"])

    def test_password_missing_lowercase(self):
        password = "UPPERCASE123!"
        self.assertEqual(IsValidPassword(password), ["Password must contain a lowercase
letter"])

    def test_password_missing_uppercase(self):
        password = "lowercase123!"
        self.assertEqual(IsValidPassword(password), ["Password must contain an
uppercase letter"])

    def test_password_missing_special_or_number(self):
        password = "NoSpecialChars"
        self.assertEqual(IsValidPassword(password), ["Password must contain a special
character or a number"])
```

```
def test_multiple_errors(self):
    password = "short"
    expected_errors = [
        "Length must be between 12 and 72 characters",
        "Password must contain an uppercase letter",
        "Password must contain a special character or a number"
    ]
    self.assertEqual(IsValidPassword(password), expected_errors)

def test_valid_password(self):
    password = "ValidPassword123!"
    self.assertEqual(IsValidPassword(password), "The password is valid")

def test_valid_password_with_min_length(self):
    password = "ValidPass123!"
    self.assertEqual(IsValidPassword(password), "The password is valid")

def test_valid_password_with_max_length(self):
    password = "A" + "a" * 70 + "1!"
    self.assertEqual(IsValidPassword(password), "The password is valid")

def test_valid_password_with_special_chars(self):
    password = "ValidPassword!@#"
    self.assertEqual(IsValidPassword(password), "The password is valid")

def test_valid_password_with_numbers(self):
    password = "ValidPassword123"
    self.assertEqual(IsValidPassword(password), "The password is valid")
```

Test suite: deepseek → Implementation: gemma

implementation_manual.py

```
import re

def IsValidPassword(password):
    if password is None or password == "":
        return ["Please fill the password"]

    errors = []

    if not (12 <= len(password) <= 72):
        errors.append("Length must be between 12 and 72 characters")

    if not re.search(r'[a-z]', password):
        errors.append("Password must contain a lowercase letter")

    if not re.search(r'[A-Z]', password):
        errors.append("Password must contain an uppercase letter")

    if not re.search(r'[!?@#$%^&*()_0-9]', password):
        # Note: The prompt specifies [!? @#$ %^&* _ 0 -9]
        # We use a regex that covers digits and the specified special characters
        if not re.search(r'[0-9!@#$%^&*_\s]', password.replace(" ", "")):
            # Re-evaluating based on the specific prompt requirement:
            # "one digit or punctuation character ([!? @#$ %^&* _ 0 -9])"
            pass

        # Let's use a more direct approach for the specific requirement
        special_or_digit_pattern = r'[a-zA-Z0-9]' # placeholder logic
        # The prompt asks for: [!? @#$ %^&* _ 0 -9]
        # This includes digits 0-9 and specific symbols.
        if not re.search(r'[0-9!@#$%^&*_\s]', password):
             # The prompt's regex is slightly ambiguous with spaces,
             # but usually implies the set of characters.
             pass

    # Refined logic to match the exact error message requirements
    errors = []

    if password is None or password == "":
        return ["Please fill the password"]

    if not (12 <= len(password) <= 72):
        errors.append("Length must be between 12 and 72 characters")

    if not any(c.islower() for c in password):
        errors.append("Password must contain a lowercase letter")

    if not any(c.isupper() for c in password):
        errors.append("Password must contain an uppercase letter")
```

```
    # Check for digit or special character [!? @#$ %^&* _ 0 -9]
    # We check if any character in the password is a digit or in the special set
    special_chars = "!@#$%^&*_"
    has_digit_or_special = any(c.isdigit() or c in special_chars for c in password)

    if not has_digit_or_special:
        errors.append("Password must contain a special character or a number")

    if errors:
        return errors

    return "The password is valid"
```

selected_test.py

```
import unittest
from implementation_manual import IsValidPassword

class TestPasswordValidator(unittest.TestCase):
    def test_missing_password(self):
        self.assertEqual(IsValidPassword(""), ["Please fill the password"])

    def test_password_too_short(self):
        self.assertEqual(IsValidPassword("Short1!"), ["Length must be between 12 and 72
characters"])

    def test_password_too_long(self):
        long_password = "a" * 73
        self.assertEqual(IsValidPassword(long_password), ["Length must be between 12
and 72 characters"])

    def test_password_missing_lowercase(self):
        password = "UPPERCASE123!"
        self.assertEqual(IsValidPassword(password), ["Password must contain a lowercase
letter"])

    def test_password_missing_uppercase(self):
        password = "lowercase123!"
        self.assertEqual(IsValidPassword(password), ["Password must contain an
uppercase letter"])

    def test_password_missing_special_or_number(self):
        password = "NoSpecialChars"
        self.assertEqual(IsValidPassword(password), ["Password must contain a special
character or a number"])

    def test_multiple_errors(self):
        password = "short"
        expected_errors = [
            "Length must be between 12 and 72 characters",
            "Password must contain an uppercase letter",
            "Password must contain a special character or a number"
        ]
```

```
    self.assertEqual(IsValidPassword(password), expected_errors)

def test_valid_password(self):
    password = "ValidPassword123!"
    self.assertEqual(IsValidPassword(password), "The password is valid")

def test_valid_password_with_min_length(self):
    password = "ValidPass123!"
    self.assertEqual(IsValidPassword(password), "The password is valid")

def test_valid_password_with_max_length(self):
    password = "A" + "a" * 70 + "1!"
    self.assertEqual(IsValidPassword(password), "The password is valid")

def test_valid_password_with_special_chars(self):
    password = "ValidPassword!@#"
    self.assertEqual(IsValidPassword(password), "The password is valid")

def test_valid_password_with_numbers(self):
    password = "ValidPassword123"
    self.assertEqual(IsValidPassword(password), "The password is valid")
```

Test suite: deepseek → Implementation: manual

implementation_manual.py

```
def IsValidPassword(password):
    errors=[]
    if len(password)==0:
        return "Please fill the password"
    if len(password)<12 or len(password)>72:
        errors.append("Length must be between 12 and 72 characters")
    lowerCase=False
    upperCase=False
    num=False
    special=False
    for char in password:
        if(char.isdigit()):
            num=True
        if(char.islower()):
            lowerCase=True
        if(char.isupper()):
            upperCase=True
        if(not char.isalnum()):
            special=True
    if not lowerCase:
        errors.append("Password must contain a lowercase letter")
    if not upperCase:
        errors.append("Password must contain an uppercase letter")
    if not special or not num:
        errors.append("Password must contain a special character or a number")
    if len(errors)==0:
        return "The password is valid"
    else:
        return errors
```

selected_test.py

```
import unittest
from implementation_manual import IsValidPassword

class TestPasswordValidator(unittest.TestCase):
    def test_missing_password(self):
        self.assertEqual(IsValidPassword(""), ["Please fill the password"])

    def test_password_too_short(self):
        self.assertEqual(IsValidPassword("Short1!"), ["Length must be between 12 and 72
characters"])

    def test_password_too_long(self):
        long_password = "a" * 73
        self.assertEqual(IsValidPassword(long_password), ["Length must be between 12
and 72 characters"])
```

```
    def test_password_missing_lowercase(self):
        password = "UPPERCASE123!"
        self.assertEqual(IsValidPassword(password), ["Password must contain a lowercase
letter"])

    def test_password_missing_uppercase(self):
        password = "lowercase123!"
        self.assertEqual(IsValidPassword(password), ["Password must contain an
uppercase letter"])

    def test_password_missing_special_or_number(self):
        password = "NoSpecialChars"
        self.assertEqual(IsValidPassword(password), ["Password must contain a special
character or a number"])

    def test_multiple_errors(self):
        password = "short"
        expected_errors = [
            "Length must be between 12 and 72 characters",
            "Password must contain an uppercase letter",
            "Password must contain a special character or a number"
        ]
        self.assertEqual(IsValidPassword(password), expected_errors)

    def test_valid_password(self):
        password = "ValidPassword123!"
        self.assertEqual(IsValidPassword(password), "The password is valid")

    def test_valid_password_with_min_length(self):
        password = "ValidPass123!"
        self.assertEqual(IsValidPassword(password), "The password is valid")

    def test_valid_password_with_max_length(self):
        password = "A" + "a" * 70 + "1!"
        self.assertEqual(IsValidPassword(password), "The password is valid")

    def test_valid_password_with_special_chars(self):
        password = "ValidPassword!@#"
        self.assertEqual(IsValidPassword(password), "The password is valid")

    def test_valid_password_with_numbers(self):
        password = "ValidPassword123"
        self.assertEqual(IsValidPassword(password), "The password is valid")
```

Test suite: deepseek → Implementation: mistral

implementation_manual.py

```
def IsValidPassword(password):
    errors = []
    if not password:
        errors.append("Please fill the password")
        return errors

    if len(password) < 12 or len(password) > 72:
        errors.append("Length must be between 12 and 72 characters")

    has_lower = any(c.islower() for c in password)
    if not has_lower:
        errors.append("Password must contain a lowercase letter")

    has_upper = any(c.isupper() for c in password)
    if not has_upper:
        errors.append("Password must contain an uppercase letter")

    has_special_or_digit = any(c.isdigit() or c in "!?@#$%^&*_-" for c in password)
    if not has_special_or_digit:
        errors.append("Password must contain a special character or a number")

    if not errors:
        return "The password is valid"
    return errors
```

selected_test.py

```
import unittest
from implementation_manual import IsValidPassword

class TestPasswordValidator(unittest.TestCase):
    def test_missing_password(self):
        self.assertEqual(IsValidPassword(""), ["Please fill the password"])

    def test_password_too_short(self):
        self.assertEqual(IsValidPassword("Short1!"), ["Length must be between 12 and 72
characters"])

    def test_password_too_long(self):
        long_password = "a" * 73
        self.assertEqual(IsValidPassword(long_password), ["Length must be between 12
and 72 characters"])

    def test_password_missing_lowercase(self):
        password = "UPPERCASE123!"
        self.assertEqual(IsValidPassword(password), ["Password must contain a lowercase
letter"])

    def test_password_missing_uppercase(self):
```

```
        password = "lowercase123!"
        self.assertEqual(IsValidPassword(password), ["Password must contain an
uppercase letter"])

    def test_password_missing_special_or_number(self):
        password = "NoSpecialChars"
        self.assertEqual(IsValidPassword(password), ["Password must contain a special
character or a number"])

    def test_multiple_errors(self):
        password = "short"
        expected_errors = [
            "Length must be between 12 and 72 characters",
            "Password must contain an uppercase letter",
            "Password must contain a special character or a number"
        ]
        self.assertEqual(IsValidPassword(password), expected_errors)

    def test_valid_password(self):
        password = "ValidPassword123!"
        self.assertEqual(IsValidPassword(password), "The password is valid")

    def test_valid_password_with_min_length(self):
        password = "ValidPass123!"
        self.assertEqual(IsValidPassword(password), "The password is valid")

    def test_valid_password_with_max_length(self):
        password = "A" + "a" * 70 + "1!"
        self.assertEqual(IsValidPassword(password), "The password is valid")

    def test_valid_password_with_special_chars(self):
        password = "ValidPassword!@#"
        self.assertEqual(IsValidPassword(password), "The password is valid")

    def test_valid_password_with_numbers(self):
        password = "ValidPassword123"
        self.assertEqual(IsValidPassword(password), "The password is valid")
```

Test suite: deepseek → Implementation: openai

implementation_manual.py

```
def IsValidPassword(password):
    errors = []
    if not password:
        errors.append("Please fill the password")
        return errors
    if not isinstance(password, str):
        errors.append("Please fill the password")
        return errors
    length = len(password)
    if length < 12 or length > 72:
        errors.append("Length must be between 12 and 72 characters")
    if not any(c.islower() for c in password):
        errors.append("Password must contain a lowercase letter")
    if not any(c.isupper() for c in password):
        errors.append("Password must contain an uppercase letter")
    if not any(c.isdigit() or c in "!@#$%^&*_?" for c in password):
        errors.append("Password must contain a special character or a number")
    if errors:
        return errors
    return "The password is valid"
```

selected_test.py

```
import unittest
from implementation_manual import IsValidPassword

class TestPasswordValidator(unittest.TestCase):
    def test_missing_password(self):
        self.assertEqual(IsValidPassword(""), ["Please fill the password"])

    def test_password_too_short(self):
        self.assertEqual(IsValidPassword("Short1!"), ["Length must be between 12 and 72
characters"])

    def test_password_too_long(self):
        long_password = "a" * 73
        self.assertEqual(IsValidPassword(long_password), ["Length must be between 12
and 72 characters"])

    def test_password_missing_lowercase(self):
        password = "UPPERCASE123!"
        self.assertEqual(IsValidPassword(password), ["Password must contain a lowercase
letter"])

    def test_password_missing_uppercase(self):
        password = "lowercase123!"
        self.assertEqual(IsValidPassword(password), ["Password must contain an
uppercase letter"])
```

```
    def test_password_missing_special_or_number(self):
        password = "NoSpecialChars"
        self.assertEqual(IsValidPassword(password), ["Password must contain a special
character or a number"])

    def test_multiple_errors(self):
        password = "short"
        expected_errors = [
            "Length must be between 12 and 72 characters",
            "Password must contain an uppercase letter",
            "Password must contain a special character or a number"
        ]
        self.assertEqual(IsValidPassword(password), expected_errors)

    def test_valid_password(self):
        password = "ValidPassword123!"
        self.assertEqual(IsValidPassword(password), "The password is valid")

    def test_valid_password_with_min_length(self):
        password = "ValidPass123!"
        self.assertEqual(IsValidPassword(password), "The password is valid")

    def test_valid_password_with_max_length(self):
        password = "A" + "a" * 70 + "1!"
        self.assertEqual(IsValidPassword(password), "The password is valid")

    def test_valid_password_with_special_chars(self):
        password = "ValidPassword!@#"
        self.assertEqual(IsValidPassword(password), "The password is valid")

    def test_valid_password_with_numbers(self):
        password = "ValidPassword123"
        self.assertEqual(IsValidPassword(password), "The password is valid")
```

Test suite: gemma → Implementation: amazon

implementation_manual.py

```
def is_valid_password(password):
    if password is None or password == "":
        return ["Please fill the password"]
    if len(password) < 12 or len(password) > 72:
        return ["Length must be between 12 and 72 characters"]
    errors = []
    has_lower = any(c.islower() for c in password)
    has_upper = any(c.isupper() for c in password)
    has_digit_or_special = any(c.isdigit() or not c.isalnum() for c in password)
    if not has_lower:
        errors.append("Password must contain a lowercase letter")
    if not has_upper:
        errors.append("Password must contain an uppercase letter")
    if not has_digit_or_special:
        errors.append("Password must contain a special character or a number")
    if errors:
        return errors
    return "The password is valid"
```

selected_test.py

```
import unittest
from implementation_manual import IsValidPassword

class TestIsValidPassword(unittest.TestCase):

    def test_valid_password_returns_success_message(self):
        # Arrange
        password = "StrongPassword123!"
        expected = "The password is valid"

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)

    def test_missing_password_returns_fill_error(self):
        # Arrange
        password = ""
        expected = ["Please fill the password"]

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)
```

```
def test_none_password_returns_fill_error(self):
    # Arrange
    password = None
    expected = ["Please fill the password"]

    # Act
    result = IsValidPassword(password)

    # Assert
    self.assertEqual(result, expected)

def test_password_too_short_returns_length_error(self):
    # Arrange
    password = "Ab1!"
    expected = ["Length must be between 12 and 72 characters"]

    # Act
    result = IsValidPassword(password)

    # Assert
    self.assertEqual(result, expected)

def test_password_too_long_returns_length_error(self):
    # Arrange
    password = "A" * 73 + "a1!"
    expected = ["Length must be between 12 and 72 characters"]

    # Act
    result = IsValidPassword(password)

    # Assert
    self.assertEqual(result, expected)

def test_missing_lowercase_returns_error(self):
    # Arrange
    password = "STRONGPASS123!"
    expected = ["Password must contain a lowercase letter"]

    # Act
    result = IsValidPassword(password)

    # Assert
    self.assertIn("Password must contain a lowercase letter", result)

def test_missing_uppercase_returns_error(self):
    # Arrange
    password = "strongpass123!"
    expected = ["Password must contain an uppercase letter"]

    # Act
    result = IsValidPassword(password)

    # Assert
```

```
        self.assertIn("Password must contain an uppercase letter", result)

    def test_missing_special_or_number_returns_error(self):
        # Arrange
        password = "StrongPassword"
        expected = ["Password must contain a special character or a number"]

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertIn("Password must contain a special character or a number", result)

    def test_multiple_validation_failures(self):
        # Arrange
        # Short, no uppercase, no special/number
        password = "abc"
        expected_errors = [
            "Length must be between 12 and 72 characters",
            "Password must contain an uppercase letter",
            "Password must contain a special character or a number"
        ]

        # Act
        result = IsValidPassword(password)

        # Assert
        for error in expected_errors:
            self.assertIn(error, result)
        self.assertEqual(len(result), 3)

    def test_boundary_length_12_is_valid(self):
        # Arrange
        password = "Abcdefghij1!" # 12 chars
        expected = "The password is valid"

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)

    def test_boundary_length_72_is_valid(self):
        # Arrange
        password = "A" * 70 + "a1!" # 72 chars
        expected = "The password is valid"

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)
```

```
if __name__ == '__main__':
    unittest.main()
```

Test suite: gemma → Implementation: deepseek

implementation_manual.py

```
def IsValidPassword(password):
    errors = []
    if not password:
        errors.append("Please fill the password")
        return errors
    if len(password) < 12 or len(password) > 72:
        errors.append("Length must be between 12 and 72 characters")
    if not any(c.islower() for c in password):
        errors.append("Password must contain a lowercase letter")
    if not any(c.isupper() for c in password):
        errors.append("Password must contain an uppercase letter")
    if not any(c.isdigit() or not c.isalnum() for c in password):
        errors.append("Password must contain a special character or a number")
    if not errors:
        return "The password is valid"
    return errors
```

selected_test.py

```
import unittest
from implementation_manual import IsValidPassword

class TestIsValidPassword(unittest.TestCase):

    def test_valid_password_returns_success_message(self):
        # Arrange
        password = "StrongPassword123!"
        expected = "The password is valid"

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)

    def test_missing_password_returns_fill_error(self):
        # Arrange
        password = ""
        expected = ["Please fill the password"]

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)

    def test_none_password_returns_fill_error(self):
        # Arrange
        password = None
```

```
        expected = ["Please fill the password"]

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)

    def test_password_too_short_returns_length_error(self):
        # Arrange
        password = "Ab1!"
        expected = ["Length must be between 12 and 72 characters"]

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)

    def test_password_too_long_returns_length_error(self):
        # Arrange
        password = "A" * 73 + "a1!"
        expected = ["Length must be between 12 and 72 characters"]

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)

    def test_missing_lowercase_returns_error(self):
        # Arrange
        password = "STRONGPASS123!"
        expected = ["Password must contain a lowercase letter"]

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertIn("Password must contain a lowercase letter", result)

    def test_missing_uppercase_returns_error(self):
        # Arrange
        password = "strongpass123!"
        expected = ["Password must contain an uppercase letter"]

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertIn("Password must contain an uppercase letter", result)

    def test_missing_special_or_number_returns_error(self):
```

```
        # Arrange
        password = "StrongPassword"
        expected = ["Password must contain a special character or a number"]

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertIn("Password must contain a special character or a number", result)

    def test_multiple_validation_failures(self):
        # Arrange
        # Short, no uppercase, no special/number
        password = "abc"
        expected_errors = [
            "Length must be between 12 and 72 characters",
            "Password must contain an uppercase letter",
            "Password must contain a special character or a number"
        ]

        # Act
        result = IsValidPassword(password)

        # Assert
        for error in expected_errors:
            self.assertIn(error, result)
        self.assertEqual(len(result), 3)

    def test_boundary_length_12_is_valid(self):
        # Arrange
        password = "Abcdefghij1!" # 12 chars
        expected = "The password is valid"

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)

    def test_boundary_length_72_is_valid(self):
        # Arrange
        password = "A" * 70 + "a1!" # 72 chars
        expected = "The password is valid"

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)

if __name__ == '__main__':
    unittest.main()
```

Test suite: gemma → Implementation: gemma

implementation_manual.py

```
import re

def IsValidPassword(password):
    if password is None or password == "":
        return ["Please fill the password"]

    errors = []

    if not (12 <= len(password) <= 72):
        errors.append("Length must be between 12 and 72 characters")

    if not re.search(r'[a-z]', password):
        errors.append("Password must contain a lowercase letter")

    if not re.search(r'[A-Z]', password):
        errors.append("Password must contain an uppercase letter")

    if not re.search(r'[!?@#$%^&*()_0-9]', password):
        # Note: The prompt specifies [!? @#$ %^&* _ 0 -9]
        # We use a regex that covers digits and the specified special characters
        if not re.search(r'[0-9!@#$%^&*_\s]', password.replace(" ", "")):
            # Re-evaluating based on the specific prompt requirement:
            # "one digit or punctuation character ([!? @#$ %^&* _ 0 -9])"
            pass

        # Let's use a more direct approach for the specific requirement
        special_or_digit_pattern = r'[a-zA-Z0-9]' # placeholder logic
        # The prompt asks for: [!? @#$ %^&* _ 0 -9]
        # This includes digits 0-9 and specific symbols.
        if not re.search(r'[0-9!@#$%^&*_\s]', password):
             # The prompt's regex is slightly ambiguous with spaces,
             # but usually implies the set of characters.
             pass

    # Refined logic to match the exact error message requirements
    errors = []

    if password is None or password == "":
        return ["Please fill the password"]

    if not (12 <= len(password) <= 72):
        errors.append("Length must be between 12 and 72 characters")

    if not any(c.islower() for c in password):
        errors.append("Password must contain a lowercase letter")

    if not any(c.isupper() for c in password):
        errors.append("Password must contain an uppercase letter")
```

```
    # Check for digit or special character [!? @#$ %^&* _ 0 -9]
    # We check if any character in the password is a digit or in the special set
    special_chars = "!@#$%^&*_"
    has_digit_or_special = any(c.isdigit() or c in special_chars for c in password)

    if not has_digit_or_special:
        errors.append("Password must contain a special character or a number")

    if errors:
        return errors

    return "The password is valid"
```

selected_test.py

```
import unittest
from implementation_manual import IsValidPassword

class TestIsValidPassword(unittest.TestCase):

    def test_valid_password_returns_success_message(self):
        # Arrange
        password = "StrongPassword123!"
        expected = "The password is valid"

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)

    def test_missing_password_returns_fill_error(self):
        # Arrange
        password = ""
        expected = ["Please fill the password"]

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)

    def test_none_password_returns_fill_error(self):
        # Arrange
        password = None
        expected = ["Please fill the password"]

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)
```

```
def test_password_too_short_returns_length_error(self):
    # Arrange
    password = "Ab1!"
    expected = ["Length must be between 12 and 72 characters"]

    # Act
    result = IsValidPassword(password)

    # Assert
    self.assertEqual(result, expected)

def test_password_too_long_returns_length_error(self):
    # Arrange
    password = "A" * 73 + "a1!"
    expected = ["Length must be between 12 and 72 characters"]

    # Act
    result = IsValidPassword(password)

    # Assert
    self.assertEqual(result, expected)

def test_missing_lowercase_returns_error(self):
    # Arrange
    password = "STRONGPASS123!"
    expected = ["Password must contain a lowercase letter"]

    # Act
    result = IsValidPassword(password)

    # Assert
    self.assertIn("Password must contain a lowercase letter", result)

def test_missing_uppercase_returns_error(self):
    # Arrange
    password = "strongpass123!"
    expected = ["Password must contain an uppercase letter"]

    # Act
    result = IsValidPassword(password)

    # Assert
    self.assertIn("Password must contain an uppercase letter", result)

def test_missing_special_or_number_returns_error(self):
    # Arrange
    password = "StrongPassword"
    expected = ["Password must contain a special character or a number"]

    # Act
    result = IsValidPassword(password)

    # Assert
```

```
        self.assertIn("Password must contain a special character or a number", result)

    def test_multiple_validation_failures(self):
        # Arrange
        # Short, no uppercase, no special/number
        password = "abc"
        expected_errors = [
            "Length must be between 12 and 72 characters",
            "Password must contain an uppercase letter",
            "Password must contain a special character or a number"
        ]

        # Act
        result = IsValidPassword(password)

        # Assert
        for error in expected_errors:
            self.assertIn(error, result)
        self.assertEqual(len(result), 3)

    def test_boundary_length_12_is_valid(self):
        # Arrange
        password = "Abcdefghij1!" # 12 chars
        expected = "The password is valid"

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)

    def test_boundary_length_72_is_valid(self):
        # Arrange
        password = "A" * 70 + "a1!" # 72 chars
        expected = "The password is valid"

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)

if __name__ == '__main__':
    unittest.main()
```

Test suite: gemma → Implementation: manual

implementation_manual.py

```
def IsValidPassword(password):
    errors=[]
    if len(password)==0:
        return "Please fill the password"
    if len(password)<12 or len(password)>72:
        errors.append("Length must be between 12 and 72 characters")
    lowerCase=False
    upperCase=False
    num=False
    special=False
    for char in password:
        if(char.isdigit()):
            num=True
        if(char.islower()):
            lowerCase=True
        if(char.isupper()):
            upperCase=True
        if(not char.isalnum()):
            special=True
    if not lowerCase:
        errors.append("Password must contain a lowercase letter")
    if not upperCase:
        errors.append("Password must contain an uppercase letter")
    if not special or not num:
        errors.append("Password must contain a special character or a number")
    if len(errors)==0:
        return "The password is valid"
    else:
        return errors
```

selected_test.py

```
import unittest
from implementation_manual import IsValidPassword

class TestIsValidPassword(unittest.TestCase):

    def test_valid_password_returns_success_message(self):
        # Arrange
        password = "StrongPassword123!"
        expected = "The password is valid"

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)
```

```
def test_missing_password_returns_fill_error(self):
    # Arrange
    password = ""
    expected = ["Please fill the password"]

    # Act
    result = IsValidPassword(password)

    # Assert
    self.assertEqual(result, expected)

def test_none_password_returns_fill_error(self):
    # Arrange
    password = None
    expected = ["Please fill the password"]

    # Act
    result = IsValidPassword(password)

    # Assert
    self.assertEqual(result, expected)

def test_password_too_short_returns_length_error(self):
    # Arrange
    password = "Ab1!"
    expected = ["Length must be between 12 and 72 characters"]

    # Act
    result = IsValidPassword(password)

    # Assert
    self.assertEqual(result, expected)

def test_password_too_long_returns_length_error(self):
    # Arrange
    password = "A" * 73 + "a1!"
    expected = ["Length must be between 12 and 72 characters"]

    # Act
    result = IsValidPassword(password)

    # Assert
    self.assertEqual(result, expected)

def test_missing_lowercase_returns_error(self):
    # Arrange
    password = "STRONGPASS123!"
    expected = ["Password must contain a lowercase letter"]

    # Act
    result = IsValidPassword(password)
```

```
        # Assert
        self.assertIn("Password must contain a lowercase letter", result)

    def test_missing_uppercase_returns_error(self):
        # Arrange
        password = "strongpass123!"
        expected = ["Password must contain an uppercase letter"]

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertIn("Password must contain an uppercase letter", result)

    def test_missing_special_or_number_returns_error(self):
        # Arrange
        password = "StrongPassword"
        expected = ["Password must contain a special character or a number"]

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertIn("Password must contain a special character or a number", result)

    def test_multiple_validation_failures(self):
        # Arrange
        # Short, no uppercase, no special/number
        password = "abc"
        expected_errors = [
            "Length must be between 12 and 72 characters",
            "Password must contain an uppercase letter",
            "Password must contain a special character or a number"
        ]

        # Act
        result = IsValidPassword(password)

        # Assert
        for error in expected_errors:
            self.assertIn(error, result)
        self.assertEqual(len(result), 3)

    def test_boundary_length_12_is_valid(self):
        # Arrange
        password = "Abcdefghij1!" # 12 chars
        expected = "The password is valid"

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)
```

```
    def test_boundary_length_72_is_valid(self):
        # Arrange
        password = "A" * 70 + "a1!" # 72 chars
        expected = "The password is valid"

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)

if __name__ == '__main__':
    unittest.main()
```

Test suite: gemma → Implementation: mistral

implementation_manual.py

```
def IsValidPassword(password):
    errors = []
    if not password:
        errors.append("Please fill the password")
        return errors

    if len(password) < 12 or len(password) > 72:
        errors.append("Length must be between 12 and 72 characters")

    has_lower = any(c.islower() for c in password)
    if not has_lower:
        errors.append("Password must contain a lowercase letter")

    has_upper = any(c.isupper() for c in password)
    if not has_upper:
        errors.append("Password must contain an uppercase letter")

    has_special_or_digit = any(c.isdigit() or c in "!?@#$%^&*_-" for c in password)
    if not has_special_or_digit:
        errors.append("Password must contain a special character or a number")

    if not errors:
        return "The password is valid"
    return errors
```

selected_test.py

```
import unittest
from implementation_manual import IsValidPassword

class TestIsValidPassword(unittest.TestCase):

    def test_valid_password_returns_success_message(self):
        # Arrange
        password = "StrongPassword123!"
        expected = "The password is valid"

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)

    def test_missing_password_returns_fill_error(self):
        # Arrange
        password = ""
        expected = ["Please fill the password"]

        # Act
```

```
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)

    def test_none_password_returns_fill_error(self):
        # Arrange
        password = None
        expected = ["Please fill the password"]

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)

    def test_password_too_short_returns_length_error(self):
        # Arrange
        password = "Ab1!"
        expected = ["Length must be between 12 and 72 characters"]

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)

    def test_password_too_long_returns_length_error(self):
        # Arrange
        password = "A" * 73 + "a1!"
        expected = ["Length must be between 12 and 72 characters"]

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)

    def test_missing_lowercase_returns_error(self):
        # Arrange
        password = "STRONGPASS123!"
        expected = ["Password must contain a lowercase letter"]

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertIn("Password must contain a lowercase letter", result)

    def test_missing_uppercase_returns_error(self):
        # Arrange
        password = "strongpass123!"
        expected = ["Password must contain an uppercase letter"]
```

```
        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertIn("Password must contain an uppercase letter", result)

    def test_missing_special_or_number_returns_error(self):
        # Arrange
        password = "StrongPassword"
        expected = ["Password must contain a special character or a number"]

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertIn("Password must contain a special character or a number", result)

    def test_multiple_validation_failures(self):
        # Arrange
        # Short, no uppercase, no special/number
        password = "abc"
        expected_errors = [
            "Length must be between 12 and 72 characters",
            "Password must contain an uppercase letter",
            "Password must contain a special character or a number"
        ]

        # Act
        result = IsValidPassword(password)

        # Assert
        for error in expected_errors:
            self.assertIn(error, result)
        self.assertEqual(len(result), 3)

    def test_boundary_length_12_is_valid(self):
        # Arrange
        password = "Abcdefghij1!" # 12 chars
        expected = "The password is valid"

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)

    def test_boundary_length_72_is_valid(self):
        # Arrange
        password = "A" * 70 + "a1!" # 72 chars
        expected = "The password is valid"

        # Act
```

```
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)

if __name__ == '__main__':
    unittest.main()
```

Test suite: gemma → Implementation: openai

implementation_manual.py

```
def IsValidPassword(password):
    errors = []
    if not password:
        errors.append("Please fill the password")
        return errors
    if not isinstance(password, str):
        errors.append("Please fill the password")
        return errors
    length = len(password)
    if length < 12 or length > 72:
        errors.append("Length must be between 12 and 72 characters")
    if not any(c.islower() for c in password):
        errors.append("Password must contain a lowercase letter")
    if not any(c.isupper() for c in password):
        errors.append("Password must contain an uppercase letter")
    if not any(c.isdigit() or c in "!@#$%^&*_?" for c in password):
        errors.append("Password must contain a special character or a number")
    if errors:
        return errors
    return "The password is valid"
```

selected_test.py

```
import unittest
from implementation_manual import IsValidPassword

class TestIsValidPassword(unittest.TestCase):

    def test_valid_password_returns_success_message(self):
        # Arrange
        password = "StrongPassword123!"
        expected = "The password is valid"

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)

    def test_missing_password_returns_fill_error(self):
        # Arrange
        password = ""
        expected = ["Please fill the password"]

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)
```

```
def test_none_password_returns_fill_error(self):
    # Arrange
    password = None
    expected = ["Please fill the password"]

    # Act
    result = IsValidPassword(password)

    # Assert
    self.assertEqual(result, expected)

def test_password_too_short_returns_length_error(self):
    # Arrange
    password = "Ab1!"
    expected = ["Length must be between 12 and 72 characters"]

    # Act
    result = IsValidPassword(password)

    # Assert
    self.assertEqual(result, expected)

def test_password_too_long_returns_length_error(self):
    # Arrange
    password = "A" * 73 + "a1!"
    expected = ["Length must be between 12 and 72 characters"]

    # Act
    result = IsValidPassword(password)

    # Assert
    self.assertEqual(result, expected)

def test_missing_lowercase_returns_error(self):
    # Arrange
    password = "STRONGPASS123!"
    expected = ["Password must contain a lowercase letter"]

    # Act
    result = IsValidPassword(password)

    # Assert
    self.assertIn("Password must contain a lowercase letter", result)

def test_missing_uppercase_returns_error(self):
    # Arrange
    password = "strongpass123!"
    expected = ["Password must contain an uppercase letter"]

    # Act
    result = IsValidPassword(password)
```

```
        # Assert
        self.assertIn("Password must contain an uppercase letter", result)

    def test_missing_special_or_number_returns_error(self):
        # Arrange
        password = "StrongPassword"
        expected = ["Password must contain a special character or a number"]

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertIn("Password must contain a special character or a number", result)

    def test_multiple_validation_failures(self):
        # Arrange
        # Short, no uppercase, no special/number
        password = "abc"
        expected_errors = [
            "Length must be between 12 and 72 characters",
            "Password must contain an uppercase letter",
            "Password must contain a special character or a number"
        ]

        # Act
        result = IsValidPassword(password)

        # Assert
        for error in expected_errors:
            self.assertIn(error, result)
        self.assertEqual(len(result), 3)

    def test_boundary_length_12_is_valid(self):
        # Arrange
        password = "Abcdefghij1!" # 12 chars
        expected = "The password is valid"

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)

    def test_boundary_length_72_is_valid(self):
        # Arrange
        password = "A" * 70 + "a1!" # 72 chars
        expected = "The password is valid"

        # Act
        result = IsValidPassword(password)

        # Assert
        self.assertEqual(result, expected)
```

```
if __name__ == '__main__':
    unittest.main()
```

Test suite: manual → Implementation: amazon

implementation_manual.py

```
def is_valid_password(password):
    if password is None or password == "":
        return ["Please fill the password"]
    if len(password) < 12 or len(password) > 72:
        return ["Length must be between 12 and 72 characters"]
    errors = []
    has_lower = any(c.islower() for c in password)
    has_upper = any(c.isupper() for c in password)
    has_digit_or_special = any(c.isdigit() or not c.isalnum() for c in password)
    if not has_lower:
        errors.append("Password must contain a lowercase letter")
    if not has_upper:
        errors.append("Password must contain an uppercase letter")
    if not has_digit_or_special:
        errors.append("Password must contain a special character or a number")
    if errors:
        return errors
    return "The password is valid"
```

selected_test.py

```
# Tests for task6_validate_password
import unittest
from implementation_manual import IsValidPassword

class Test_password_valid(unittest.TestCase):
    def test_no_password(self):
        self.assertEqual(IsValidPassword(""),"Please fill the password")

    def test_no_lowercase(self):
        self.assertEqual(IsValidPassword("ABCDEFGHIJK*1"),['Password must contain a
lowercase letter'])

    def test_no_upprcase(self):
        self.assertEqual(IsValidPassword("abcdefghijk*1"),['Password must contain an
uppercase letter'])

    def test_no_symbols(self):
        self.assertEqual(IsValidPassword("Abcdefghijkl1"),['Password must contain a
special character or a number'])

    def test_lower_upper_no_symbol(self):
        self.assertEqual(IsValidPassword("Abcdefghijkl1"),['Password must contain a
special character or a number'])

    def test_nolower_upper_symbol(self):
        self.assertEqual(IsValidPassword("ABCDEFGHIJK*1"),['Password must contain a
lowercase letter'])
```

```
    def test_noupper_lower_symbol(self):
        self.assertEqual(IsValidPassword("abcdefghijk*1"),['Password must contain an
uppercase letter'])

    def test_diff_language(self):
        self.assertEqual(IsValidPassword("ಅಆಇಅಆಇಅಆಇ1Aa*"),"The password is valid")

    def test_short_len_pwd(self):
        self.assertEqual(IsValidPassword("Shean123*"),['Length must be between 12 and
72 characters'])

    def test_long_len_pwd(self):
        self.assertEqual(IsValidPassword("Shean123*" + "a"*70),['Length must be between
12 and 72 characters'])

    def test_working_pwd(self):
        self.assertEqual(IsValidPassword("Shean12354*ab"),"The password is valid")

    def test_mix_languages(self):
        self.assertEqual(IsValidPassword("ಅಆಇಅಆಇ1231oremipsumZZ*"),"The password is
valid")

    def test_only_symbols(self):
        self.assertEqual(IsValidPassword("!@#$%^&*()*!@"),['Password must contain a
lowercase letter','Password must contain an uppercase letter','Password must contain a
special character or a number'])

    def test_spaces_in_pwd(self):
        self.assertEqual(IsValidPassword("Shean 12354*ab"),"The password is valid")

if __name__ == "__main__":
    unittest.main()
```

Test suite: manual → Implementation: deepseek

implementation_manual.py

```
def IsValidPassword(password):
    errors = []
    if not password:
        errors.append("Please fill the password")
        return errors
    if len(password) < 12 or len(password) > 72:
        errors.append("Length must be between 12 and 72 characters")
    if not any(c.islower() for c in password):
        errors.append("Password must contain a lowercase letter")
    if not any(c.isupper() for c in password):
        errors.append("Password must contain an uppercase letter")
    if not any(c.isdigit() or not c.isalnum() for c in password):
        errors.append("Password must contain a special character or a number")
    if not errors:
        return "The password is valid"
    return errors
```

selected_test.py

```
# Tests for task6_validate_password
import unittest
from implementation_manual import IsValidPassword

class Test_password_valid(unittest.TestCase):
    def test_no_password(self):
        self.assertEqual(IsValidPassword(""),"Please fill the password")

    def test_no_lowercase(self):
        self.assertEqual(IsValidPassword("ABCDEFGHIJK*1"),['Password must contain a
lowercase letter'])

    def test_no_upprcase(self):
        self.assertEqual(IsValidPassword("abcdefghijk*1"),['Password must contain an
uppercase letter'])

    def test_no_symbols(self):
        self.assertEqual(IsValidPassword("Abcdefghijkl1"),['Password must contain a
special character or a number'])

    def test_lower_upper_no_symbol(self):
        self.assertEqual(IsValidPassword("Abcdefghijkl1"),['Password must contain a
special character or a number'])

    def test_nolower_upper_symbol(self):
        self.assertEqual(IsValidPassword("ABCDEFGHIJK*1"),['Password must contain a
lowercase letter'])

    def test_noupper_lower_symbol(self):
        self.assertEqual(IsValidPassword("abcdefghijk*1"),['Password must contain an
```

```
uppercase letter'])

    def test_diff_language(self):
        self.assertEqual(IsValidPassword("ಅಆಇಈಉಊಋಎಏ12ಐ"),"The password is valid")

    def test_short_len_pwd(self):
        self.assertEqual(IsValidPassword("Shean123*"),['Length must be between 12 and
72 characters'])

    def test_long_len_pwd(self):
        self.assertEqual(IsValidPassword("Shean123*" + "a"*70),['Length must be between
12 and 72 characters'])

    def test_working_pwd(self):
        self.assertEqual(IsValidPassword("Shean12354*ab"),"The password is valid")

    def test_mix_languages(self):
        self.assertEqual(IsValidPassword("ಅಆಇಈ1231oremipsumZZ*"),"The password is
valid")

    def test_only_symbols(self):
        self.assertEqual(IsValidPassword("!@#$%^&*()*!@"),['Password must contain a
lowercase letter','Password must contain an uppercase letter','Password must contain a
special character or a number'])

    def test_spaces_in_pwd(self):
        self.assertEqual(IsValidPassword("Shean 12354*ab"),"The password is valid")

if __name__ == "__main__":
    unittest.main()
```

Test suite: manual → Implementation: gemma

implementation_manual.py

```
import re

def IsValidPassword(password):
    if password is None or password == "":
        return ["Please fill the password"]

    errors = []

    if not (12 <= len(password) <= 72):
        errors.append("Length must be between 12 and 72 characters")

    if not re.search(r'[a-z]', password):
        errors.append("Password must contain a lowercase letter")

    if not re.search(r'[A-Z]', password):
        errors.append("Password must contain an uppercase letter")

    if not re.search(r'[!?@#$%^&*()_0-9]', password):
        # Note: The prompt specifies [!? @#$ %^&* _ 0 -9]
        # We use a regex that covers digits and the specified special characters
        if not re.search(r'[0-9!@#$%^&*_\s]', password.replace(" ", "")):
            # Re-evaluating based on the specific prompt requirement:
            # "one digit or punctuation character ([!? @#$ %^&* _ 0 -9])"
            pass

        # Let's use a more direct approach for the specific requirement
        special_or_digit_pattern = r'[a-zA-Z0-9]' # placeholder logic
        # The prompt asks for: [!? @#$ %^&* _ 0 -9]
        # This includes digits 0-9 and specific symbols.
        if not re.search(r'[0-9!@#$%^&*_\s]', password):
             # The prompt's regex is slightly ambiguous with spaces,
             # but usually implies the set of characters.
             pass

    # Refined logic to match the exact error message requirements
    errors = []

    if password is None or password == "":
        return ["Please fill the password"]

    if not (12 <= len(password) <= 72):
        errors.append("Length must be between 12 and 72 characters")

    if not any(c.islower() for c in password):
        errors.append("Password must contain a lowercase letter")

    if not any(c.isupper() for c in password):
        errors.append("Password must contain an uppercase letter")
```

```
    # Check for digit or special character [!? @#$ %^&* _ 0 -9]
    # We check if any character in the password is a digit or in the special set
    special_chars = "!@#$%^&*_"
    has_digit_or_special = any(c.isdigit() or c in special_chars for c in password)

    if not has_digit_or_special:
        errors.append("Password must contain a special character or a number")

    if errors:
        return errors

    return "The password is valid"
```

selected_test.py

```
# Tests for task6_validate_password
import unittest
from implementation_manual import IsValidPassword

class Test_password_valid(unittest.TestCase):
    def test_no_password(self):
        self.assertEqual(IsValidPassword(""),"Please fill the password")

    def test_no_lowercase(self):
        self.assertEqual(IsValidPassword("ABCDEFGHIJK*1"),['Password must contain a
lowercase letter'])

    def test_no_upprcase(self):
        self.assertEqual(IsValidPassword("abcdefghijk*1"),['Password must contain an
uppercase letter'])

    def test_no_symbols(self):
        self.assertEqual(IsValidPassword("Abcdefghijkl1"),['Password must contain a
special character or a number'])

    def test_lower_upper_no_symbol(self):
        self.assertEqual(IsValidPassword("Abcdefghijkl1"),['Password must contain a
special character or a number'])

    def test_nolower_upper_symbol(self):
        self.assertEqual(IsValidPassword("ABCDEFGHIJK*1"),['Password must contain a
lowercase letter'])

    def test_noupper_lower_symbol(self):
        self.assertEqual(IsValidPassword("abcdefghijk*1"),['Password must contain an
uppercase letter'])

    def test_diff_language(self):
        self.assertEqual(IsValidPassword("ಅಆಇಅಆಇಅಆ1ಅ*"),"The password is valid")

    def test_short_len_pwd(self):
        self.assertEqual(IsValidPassword("Shean123*"),['Length must be between 12 and
72 characters'])
```

```
    def test_long_len_pwd(self):
        self.assertEqual(IsValidPassword("Shean123*" + "a"*70),['Length must be between
12 and 72 characters'])

    def test_working_pwd(self):
        self.assertEqual(IsValidPassword("Shean12354*ab"),"The password is valid")

    def test_mix_languages(self):
        self.assertEqual(IsValidPassword("ಅಆಇಈ1231oremipsumZZ*"),"The password is
valid")

    def test_only_symbols(self):
        self.assertEqual(IsValidPassword("!@#$%^&*()*!@"),['Password must contain a
lowercase letter','Password must contain an uppercase letter','Password must contain a
special character or a number'])

    def test_spaces_in_pwd(self):
        self.assertEqual(IsValidPassword("Shean 12354*ab"),"The password is valid")

if __name__ == "__main__":
    unittest.main()
```

Test suite: manual → Implementation: manual

implementation_manual.py

```
def IsValidPassword(password):
    errors=[]
    if len(password)==0:
        return "Please fill the password"
    if len(password)<12 or len(password)>72:
        errors.append("Length must be between 12 and 72 characters")
    lowerCase=False
    upperCase=False
    num=False
    special=False
    for char in password:
        if(char.isdigit()):
            num=True
        if(char.islower()):
            lowerCase=True
        if(char.isupper()):
            upperCase=True
        if(not char.isalnum()):
            special=True
    if not lowerCase:
        errors.append("Password must contain a lowercase letter")
    if not upperCase:
        errors.append("Password must contain an uppercase letter")
    if not special or not num:
        errors.append("Password must contain a special character or a number")
    if len(errors)==0:
        return "The password is valid"
    else:
        return errors
```

selected_test.py

```
# Tests for task6_validate_password
import unittest
from implementation_manual import IsValidPassword

class Test_password_valid(unittest.TestCase):
    def test_no_password(self):
        self.assertEqual(IsValidPassword(""),"Please fill the password")

    def test_no_lowercase(self):
        self.assertEqual(IsValidPassword("ABCDEFGHIJK*1"),['Password must contain a
lowercase letter'])

    def test_no_upprcase(self):
        self.assertEqual(IsValidPassword("abcdefghijk*1"),['Password must contain an
uppercase letter'])
```

```
    def test_no_symbols(self):
        self.assertEqual(IsValidPassword("Abcdefghijkl1"),['Password must contain a
special character or a number'])

    def test_lower_upper_no_symbol(self):
        self.assertEqual(IsValidPassword("Abcdefghijkl1"),['Password must contain a
special character or a number'])

    def test_nolower_upper_symbol(self):
        self.assertEqual(IsValidPassword("ABCDEFGHIJK*1"),['Password must contain a
lowercase letter'])

    def test_noupper_lower_symbol(self):
        self.assertEqual(IsValidPassword("abcdefghijk*1"),['Password must contain an
uppercase letter'])

    def test_diff_language(self):
        self.assertEqual(IsValidPassword("ಅಆಇಈಉಊಋ1Aa"),"The password is valid")

    def test_short_len_pwd(self):
        self.assertEqual(IsValidPassword("Shean123*"),['Length must be between 12 and
72 characters'])

    def test_long_len_pwd(self):
        self.assertEqual(IsValidPassword("Shean123*" + "a"*70),['Length must be between
12 and 72 characters'])

    def test_working_pwd(self):
        self.assertEqual(IsValidPassword("Shean12354*ab"),"The password is valid")

    def test_mix_languages(self):
        self.assertEqual(IsValidPassword("ಅಆಇಈ1231LoremipsumZZ*"),"The password is
valid")

    def test_only_symbols(self):
        self.assertEqual(IsValidPassword("!@#$%^&*()*!@"),['Password must contain a
lowercase letter','Password must contain an uppercase letter','Password must contain a
special character or a number'])

    def test_spaces_in_pwd(self):
        self.assertEqual(IsValidPassword("Shean 12354*ab"),"The password is valid")

if __name__ == "__main__":
    unittest.main()
```

Test suite: manual → Implementation: mistral

implementation_manual.py

```
def IsValidPassword(password):
    errors = []
    if not password:
        errors.append("Please fill the password")
        return errors

    if len(password) < 12 or len(password) > 72:
        errors.append("Length must be between 12 and 72 characters")

    has_lower = any(c.islower() for c in password)
    if not has_lower:
        errors.append("Password must contain a lowercase letter")

    has_upper = any(c.isupper() for c in password)
    if not has_upper:
        errors.append("Password must contain an uppercase letter")

    has_special_or_digit = any(c.isdigit() or c in "!?@#$%^&*_-" for c in password)
    if not has_special_or_digit:
        errors.append("Password must contain a special character or a number")

    if not errors:
        return "The password is valid"
    return errors
```

selected_test.py

```
# Tests for task6_validate_password
import unittest
from implementation_manual import IsValidPassword

class Test_password_valid(unittest.TestCase):
    def test_no_password(self):
        self.assertEqual(IsValidPassword(""),"Please fill the password")

    def test_no_lowercase(self):
        self.assertEqual(IsValidPassword("ABCDEFGHIJK*1"),['Password must contain a
lowercase letter'])

    def test_no_upprcase(self):
        self.assertEqual(IsValidPassword("abcdefghijk*1"),['Password must contain an
uppercase letter'])

    def test_no_symbols(self):
        self.assertEqual(IsValidPassword("Abcdefghijkl1"),['Password must contain a
special character or a number'])

    def test_lower_upper_no_symbol(self):
        self.assertEqual(IsValidPassword("Abcdefghijkl1"),['Password must contain a
```

```
special character or a number'])

    def test_nolower_upper_symbol(self):
        self.assertEqual(IsValidPassword("ABCDEFGHIJK*1"),['Password must contain a
lowercase letter'])

    def test_noupper_lower_symbol(self):
        self.assertEqual(IsValidPassword("abcdefghijk*1"),['Password must contain an
uppercase letter'])

    def test_diff_language(self):
        self.assertEqual(IsValidPassword("ಅಆಇಈಉಊಋ1Aa"),"The password is valid")

    def test_short_len_pwd(self):
        self.assertEqual(IsValidPassword("Shean123*"),['Length must be between 12 and
72 characters'])

    def test_long_len_pwd(self):
        self.assertEqual(IsValidPassword("Shean123*" + "a"*70),['Length must be between
12 and 72 characters'])

    def test_working_pwd(self):
        self.assertEqual(IsValidPassword("Shean12354*ab"),"The password is valid")

    def test_mix_languages(self):
        self.assertEqual(IsValidPassword("ಅಆಇಈ1231oremipsumZZ*"),"The password is
valid")

    def test_only_symbols(self):
        self.assertEqual(IsValidPassword("!@#$%^&*()*!@"),['Password must contain a
lowercase letter','Password must contain an uppercase letter','Password must contain a
special character or a number'])

    def test_spaces_in_pwd(self):
        self.assertEqual(IsValidPassword("Shean 12354*ab"),"The password is valid")

if __name__ == "__main__":
    unittest.main()
```

Test suite: manual → Implementation: openai

implementation_manual.py

```
def IsValidPassword(password):
    errors = []
    if not password:
        errors.append("Please fill the password")
        return errors
    if not isinstance(password, str):
        errors.append("Please fill the password")
        return errors
    length = len(password)
    if length < 12 or length > 72:
        errors.append("Length must be between 12 and 72 characters")
    if not any(c.islower() for c in password):
        errors.append("Password must contain a lowercase letter")
    if not any(c.isupper() for c in password):
        errors.append("Password must contain an uppercase letter")
    if not any(c.isdigit() or c in "!@#$%^&*_?" for c in password):
        errors.append("Password must contain a special character or a number")
    if errors:
        return errors
    return "The password is valid"
```

selected_test.py

```
# Tests for task6_validate_password
import unittest
from implementation_manual import IsValidPassword

class Test_password_valid(unittest.TestCase):
    def test_no_password(self):
        self.assertEqual(IsValidPassword(""),"Please fill the password")

    def test_no_lowercase(self):
        self.assertEqual(IsValidPassword("ABCDEFGHIJK*1"),['Password must contain a
lowercase letter'])

    def test_no_upprcase(self):
        self.assertEqual(IsValidPassword("abcdefghijk*1"),['Password must contain an
uppercase letter'])

    def test_no_symbols(self):
        self.assertEqual(IsValidPassword("Abcdefghijkl1"),['Password must contain a
special character or a number'])

    def test_lower_upper_no_symbol(self):
        self.assertEqual(IsValidPassword("Abcdefghijkl1"),['Password must contain a
special character or a number'])

    def test_nolower_upper_symbol(self):
        self.assertEqual(IsValidPassword("ABCDEFGHIJK*1"),['Password must contain a
```

```
lowercase letter'])

    def test_noupper_lower_symbol(self):
        self.assertEqual(IsValidPassword("abcdefghijk*1"),['Password must contain an
uppercase letter'])

    def test_diff_language(self):
        self.assertEqual(IsValidPassword("ᄋᄋᄒᄋᄋᄒᄋᄋ1A*"),"The password is valid")

    def test_short_len_pwd(self):
        self.assertEqual(IsValidPassword("Shean123*"),['Length must be between 12 and
72 characters'])

    def test_long_len_pwd(self):
        self.assertEqual(IsValidPassword("Shean123*" + "a"*70),['Length must be between
12 and 72 characters'])

    def test_working_pwd(self):
        self.assertEqual(IsValidPassword("Shean12354*ab"),"The password is valid")

    def test_mix_languages(self):
        self.assertEqual(IsValidPassword("ᄋᄋᄒᄋ1231oremipsumZZ*"),"The password is
valid")

    def test_only_symbols(self):
        self.assertEqual(IsValidPassword("!@#$%^&*()*!@"),['Password must contain a
lowercase letter','Password must contain an uppercase letter','Password must contain a
special character or a number'])

    def test_spaces_in_pwd(self):
        self.assertEqual(IsValidPassword("Shean 12354*ab"),"The password is valid")

if __name__ == "__main__":
    unittest.main()
```

Test suite: mistral → Implementation: amazon

implementation_manual.py

```
def is_valid_password(password):
    if password is None or password == "":
        return ["Please fill the password"]
    if len(password) < 12 or len(password) > 72:
        return ["Length must be between 12 and 72 characters"]
    errors = []
    has_lower = any(c.islower() for c in password)
    has_upper = any(c.isupper() for c in password)
    has_digit_or_special = any(c.isdigit() or not c.isalnum() for c in password)
    if not has_lower:
        errors.append("Password must contain a lowercase letter")
    if not has_upper:
        errors.append("Password must contain an uppercase letter")
    if not has_digit_or_special:
        errors.append("Password must contain a special character or a number")
    if errors:
        return errors
    return "The password is valid"
```

selected_test.py

```
from implementation_manual import IsValidPassword
import unittest

class TestIsValidPassword(unittest.TestCase):

    def test_missing_password(self):
        result = IsValidPassword("")
        self.assertEqual(result, ["Please fill the password"])

    def test_password_none(self):
        result = IsValidPassword(None)
        self.assertEqual(result, ["Please fill the password"])

    def test_password_too_short(self):
        result = IsValidPassword("short")
        self.assertEqual(result, ["Length must be between 12 and 72 characters"])

    def test_password_too_long(self):
        result = IsValidPassword("a" * 73)
        self.assertEqual(result, ["Length must be between 12 and 72 characters"])

    def test_password_exact_min_length(self):
        result = IsValidPassword("Aa1!aaaaaaaaa")
        self.assertEqual(result, "The password is valid")

    def test_password_exact_max_length(self):
        result =
```

```
IsValidPassword("Aa1!aaaa
```

Test suite: mistral → Implementation: deepseek

implementation_manual.py

```
def IsValidPassword(password):
    errors = []
    if not password:
        errors.append("Please fill the password")
        return errors
    if len(password) < 12 or len(password) > 72:
        errors.append("Length must be between 12 and 72 characters")
    if not any(c.islower() for c in password):
        errors.append("Password must contain a lowercase letter")
    if not any(c.isupper() for c in password):
        errors.append("Password must contain an uppercase letter")
    if not any(c.isdigit() or not c.isalnum() for c in password):
        errors.append("Password must contain a special character or a number")
    if not errors:
        return "The password is valid"
    return errors
```

selected_test.py

```
from implementation_manual import IsValidPassword
import unittest

class TestIsValidPassword(unittest.TestCase):

    def test_missing_password(self):
        result = IsValidPassword("")
        self.assertEqual(result, ["Please fill the password"])

    def test_password_none(self):
        result = IsValidPassword(None)
        self.assertEqual(result, ["Please fill the password"])

    def test_password_too_short(self):
        result = IsValidPassword("short")
        self.assertEqual(result, ["Length must be between 12 and 72 characters"])

    def test_password_too_long(self):
        result = IsValidPassword("a" * 73)
        self.assertEqual(result, ["Length must be between 12 and 72 characters"])

    def test_password_exact_min_length(self):
        result = IsValidPassword("Aa1!aaaaaaaaa")
        self.assertEqual(result, "The password is valid")

    def test_password_exact_max_length(self):
        result =
IsValidPassword("Aa1!aaaaaaaa
```

```
    def test_password_missing_lowercase(self):
        result = IsValidPassword("AAAAAAAAAAA1!")
        self.assertEqual(result, ["Password must contain a lowercase letter"])

    def test_password_missing_uppercase(self):
        result = IsValidPassword("aaaaaaaaaaa1!")
        self.assertEqual(result, ["Password must contain an uppercase letter"])

    def test_password_missing_special_or_number(self):
        result = IsValidPassword("Aaaaaaaaaaaaa")
        self.assertEqual(result, ["Password must contain a special character or a
number"])

    def test_password_missing_multiple_requirements(self):
        result = IsValidPassword("short")
        self.assertEqual(result, [
            "Length must be between 12 and 72 characters",
            "Password must contain an uppercase letter",
            "Password must contain a special character or a number"
        ])

    def test_password_missing_all_requirements(self):
        result = IsValidPassword("12345")
        self.assertEqual(result, [
            "Length must be between 12 and 72 characters",
            "Password must contain an uppercase letter",
            "Password must contain a lowercase letter",
            "Password must contain a special character or a number"
        ])

    def test_valid_password_with_all_requirements(self):
        result = IsValidPassword("ValidPass123!")
        self.assertEqual(result, "The password is valid")

    def test_valid_password_with_special_chars(self):
        result = IsValidPassword("Valid_Pass@123")
        self.assertEqual(result, "The password is valid")

    def test_password_with_spaces(self):
        result = IsValidPassword(" Valid Pass123! ")
        self.assertEqual(result, "The password is valid")

    def test_password_with_only_special_chars(self):
        result = IsValidPassword("!@#$%^&*()_+")
        self.assertEqual(result, [
            "Length must be between 12 and 72 characters",
            "Password must contain an uppercase letter",
            "Password must contain a lowercase letter"
        ])
```

Test suite: mistral → Implementation: gemma

implementation_manual.py

```
import re

def IsValidPassword(password):
    if password is None or password == "":
        return ["Please fill the password"]

    errors = []

    if not (12 <= len(password) <= 72):
        errors.append("Length must be between 12 and 72 characters")

    if not re.search(r'[a-z]', password):
        errors.append("Password must contain a lowercase letter")

    if not re.search(r'[A-Z]', password):
        errors.append("Password must contain an uppercase letter")

    if not re.search(r'[!?@#$%^&*()_0-9]', password):
        # Note: The prompt specifies [!? @#$ %^&* _ 0 -9]
        # We use a regex that covers digits and the specified special characters
        if not re.search(r'[0-9!@#$%^&*_\s]', password.replace(" ", "")):
            # Re-evaluating based on the specific prompt requirement:
            # "one digit or punctuation character ([!? @#$ %^&* _ 0 -9])"
            pass

        # Let's use a more direct approach for the specific requirement
        special_or_digit_pattern = r'[a-zA-Z0-9]' # placeholder logic
        # The prompt asks for: [!? @#$ %^&* _ 0 -9]
        # This includes digits 0-9 and specific symbols.
        if not re.search(r'[0-9!@#$%^&*_\s]', password):
             # The prompt's regex is slightly ambiguous with spaces,
             # but usually implies the set of characters.
             pass

    # Refined logic to match the exact error message requirements
    errors = []

    if password is None or password == "":
        return ["Please fill the password"]

    if not (12 <= len(password) <= 72):
        errors.append("Length must be between 12 and 72 characters")

    if not any(c.islower() for c in password):
        errors.append("Password must contain a lowercase letter")

    if not any(c.isupper() for c in password):
        errors.append("Password must contain an uppercase letter")
```

```
    # Check for digit or special character [!? @#$ %^&* _ 0 -9]
    # We check if any character in the password is a digit or in the special set
    special_chars = "!@#$%^&*_"
    has_digit_or_special = any(c.isdigit() or c in special_chars for c in password)

    if not has_digit_or_special:
        errors.append("Password must contain a special character or a number")

    if errors:
        return errors

    return "The password is valid"
```

selected_test.py

```
from implementation_manual import IsValidPassword
import unittest

class TestIsValidPassword(unittest.TestCase):

    def test_missing_password(self):
        result = IsValidPassword("")
        self.assertEqual(result, ["Please fill the password"])

    def test_password_none(self):
        result = IsValidPassword(None)
        self.assertEqual(result, ["Please fill the password"])

    def test_password_too_short(self):
        result = IsValidPassword("short")
        self.assertEqual(result, ["Length must be between 12 and 72 characters"])

    def test_password_too_long(self):
        result = IsValidPassword("a" * 73)
        self.assertEqual(result, ["Length must be between 12 and 72 characters"])

    def test_password_exact_min_length(self):
        result = IsValidPassword("Aa1!aaaaaaaaa")
        self.assertEqual(result, "The password is valid")

    def test_password_exact_max_length(self):
        result =
IsValidPassword("Aa1!aaaaaaaa
```

```
    def test_password_missing_special_or_number(self):
        result = IsValidPassword("Aaaaaaaaaaaaa")
        self.assertEqual(result, ["Password must contain a special character or a
number"])

    def test_password_missing_multiple_requirements(self):
        result = IsValidPassword("short")
        self.assertEqual(result, [
            "Length must be between 12 and 72 characters",
            "Password must contain an uppercase letter",
            "Password must contain a special character or a number"
        ])

    def test_password_missing_all_requirements(self):
        result = IsValidPassword("12345")
        self.assertEqual(result, [
            "Length must be between 12 and 72 characters",
            "Password must contain an uppercase letter",
            "Password must contain a lowercase letter",
            "Password must contain a special character or a number"
        ])

    def test_valid_password_with_all_requirements(self):
        result = IsValidPassword("ValidPass123!")
        self.assertEqual(result, "The password is valid")

    def test_valid_password_with_special_chars(self):
        result = IsValidPassword("Valid_Pass@123")
        self.assertEqual(result, "The password is valid")

    def test_password_with_spaces(self):
        result = IsValidPassword(" Valid Pass123! ")
        self.assertEqual(result, "The password is valid")

    def test_password_with_only_special_chars(self):
        result = IsValidPassword("!@#$%^&*()_+")
        self.assertEqual(result, [
            "Length must be between 12 and 72 characters",
            "Password must contain an uppercase letter",
            "Password must contain a lowercase letter"
        ])
```

Test suite: mistral → Implementation: manual

implementation_manual.py

```
def IsValidPassword(password):
    errors=[]
    if len(password)==0:
        return "Please fill the password"
    if len(password)<12 or len(password)>72:
        errors.append("Length must be between 12 and 72 characters")
    lowerCase=False
    upperCase=False
    num=False
    special=False
    for char in password:
        if(char.isdigit()):
            num=True
        if(char.islower()):
            lowerCase=True
        if(char.isupper()):
            upperCase=True
        if(not char.isalnum()):
            special=True
    if not lowerCase:
        errors.append("Password must contain a lowercase letter")
    if not upperCase:
        errors.append("Password must contain an uppercase letter")
    if not special or not num:
        errors.append("Password must contain a special character or a number")
    if len(errors)==0:
        return "The password is valid"
    else:
        return errors
```

selected_test.py

```
from implementation_manual import IsValidPassword
import unittest

class TestIsValidPassword(unittest.TestCase):

    def test_missing_password(self):
        result = IsValidPassword("")
        self.assertEqual(result, ["Please fill the password"])

    def test_password_none(self):
        result = IsValidPassword(None)
        self.assertEqual(result, ["Please fill the password"])

    def test_password_too_short(self):
        result = IsValidPassword("short")
```

```
        self.assertEqual(result, ["Length must be between 12 and 72 characters"])

    def test_password_too_long(self):
        result = IsValidPassword("a" * 73)
        self.assertEqual(result, ["Length must be between 12 and 72 characters"])

    def test_password_exact_min_length(self):
        result = IsValidPassword("Aa1!aaaaaaaaa")
        self.assertEqual(result, "The password is valid")

    def test_password_exact_max_length(self):
        result =
IsValidPassword("Aa1!aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa")
        self.assertEqual(result, "The password is valid")

    def test_password_missing_lowercase(self):
        result = IsValidPassword("AAAAAAAAAAAA1!")
        self.assertEqual(result, ["Password must contain a lowercase letter"])

    def test_password_missing_uppercase(self):
        result = IsValidPassword("aaaaaaaaaaaa1!")
        self.assertEqual(result, ["Password must contain an uppercase letter"])

    def test_password_missing_special_or_number(self):
        result = IsValidPassword("Aaaaaaaaaaaaaa")
        self.assertEqual(result, ["Password must contain a special character or a
number"])

    def test_password_missing_multiple_requirements(self):
        result = IsValidPassword("short")
        self.assertEqual(result, [
            "Length must be between 12 and 72 characters",
            "Password must contain an uppercase letter",
            "Password must contain a special character or a number"
        ])

    def test_password_missing_all_requirements(self):
        result = IsValidPassword("12345")
        self.assertEqual(result, [
            "Length must be between 12 and 72 characters",
            "Password must contain an uppercase letter",
            "Password must contain a lowercase letter",
            "Password must contain a special character or a number"
        ])

    def test_valid_password_with_all_requirements(self):
        result = IsValidPassword("ValidPass123!")
        self.assertEqual(result, "The password is valid")

    def test_valid_password_with_special_chars(self):
        result = IsValidPassword("Valid_Pass@123")
        self.assertEqual(result, "The password is valid")
```

```
def test_password_with_spaces(self):
    result = IsValidPassword(" Valid Pass123! ")
    self.assertEqual(result, "The password is valid")

def test_password_with_only_special_chars(self):
    result = IsValidPassword("!@#$%^&*()_+")
    self.assertEqual(result, [
        "Length must be between 12 and 72 characters",
        "Password must contain an uppercase letter",
        "Password must contain a lowercase letter"
    ])
```

Test suite: mistral → Implementation: mistral

implementation_manual.py

```
def IsValidPassword(password):
    errors = []
    if not password:
        errors.append("Please fill the password")
        return errors

    if len(password) < 12 or len(password) > 72:
        errors.append("Length must be between 12 and 72 characters")

    has_lower = any(c.islower() for c in password)
    if not has_lower:
        errors.append("Password must contain a lowercase letter")

    has_upper = any(c.isupper() for c in password)
    if not has_upper:
        errors.append("Password must contain an uppercase letter")

    has_special_or_digit = any(c.isdigit() or c in "!?@#$%^&*_-" for c in password)
    if not has_special_or_digit:
        errors.append("Password must contain a special character or a number")

    if not errors:
        return "The password is valid"
    return errors
```

selected_test.py

```
from implementation_manual import IsValidPassword
import unittest

class TestIsValidPassword(unittest.TestCase):

    def test_missing_password(self):
        result = IsValidPassword("")
        self.assertEqual(result, ["Please fill the password"])

    def test_password_none(self):
        result = IsValidPassword(None)
        self.assertEqual(result, ["Please fill the password"])

    def test_password_too_short(self):
        result = IsValidPassword("short")
        self.assertEqual(result, ["Length must be between 12 and 72 characters"])

    def test_password_too_long(self):
        result = IsValidPassword("a" * 73)
        self.assertEqual(result, ["Length must be between 12 and 72 characters"])

    def test_password_exact_min_length(self):
```

```
        result = IsValidPassword("Aa1!aaaaaaaaa")
        self.assertEqual(result, "The password is valid")

    def test_password_exact_max_length(self):
        result =
IsValidPassword("Aa1!aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa")
        self.assertEqual(result, "The password is valid")

    def test_password_missing_lowercase(self):
        result = IsValidPassword("AAAAAAAAAAA1!")
        self.assertEqual(result, ["Password must contain a lowercase letter"])

    def test_password_missing_uppercase(self):
        result = IsValidPassword("aaaaaaaaaaa1!")
        self.assertEqual(result, ["Password must contain an uppercase letter"])

    def test_password_missing_special_or_number(self):
        result = IsValidPassword("Aaaaaaaaaaaaa")
        self.assertEqual(result, ["Password must contain a special character or a
number"])

    def test_password_missing_multiple_requirements(self):
        result = IsValidPassword("short")
        self.assertEqual(result, [
            "Length must be between 12 and 72 characters",
            "Password must contain an uppercase letter",
            "Password must contain a special character or a number"
        ])

    def test_password_missing_all_requirements(self):
        result = IsValidPassword("12345")
        self.assertEqual(result, [
            "Length must be between 12 and 72 characters",
            "Password must contain an uppercase letter",
            "Password must contain a lowercase letter",
            "Password must contain a special character or a number"
        ])

    def test_valid_password_with_all_requirements(self):
        result = IsValidPassword("ValidPass123!")
        self.assertEqual(result, "The password is valid")

    def test_valid_password_with_special_chars(self):
        result = IsValidPassword("Valid_Pass@123")
        self.assertEqual(result, "The password is valid")

    def test_password_with_spaces(self):
        result = IsValidPassword(" Valid Pass123! ")
        self.assertEqual(result, "The password is valid")

    def test_password_with_only_special_chars(self):
        result = IsValidPassword("!@#$%^&*()_+")
        self.assertEqual(result, [
```

```
    "Length must be between 12 and 72 characters",
    "Password must contain an uppercase letter",
    "Password must contain a lowercase letter"
])
```

Test suite: mistral → Implementation: openai

implementation_manual.py

```
def IsValidPassword(password):
    errors = []
    if not password:
        errors.append("Please fill the password")
        return errors
    if not isinstance(password, str):
        errors.append("Please fill the password")
        return errors
    length = len(password)
    if length < 12 or length > 72:
        errors.append("Length must be between 12 and 72 characters")
    if not any(c.islower() for c in password):
        errors.append("Password must contain a lowercase letter")
    if not any(c.isupper() for c in password):
        errors.append("Password must contain an uppercase letter")
    if not any(c.isdigit() or c in "!@#$%^&*_?" for c in password):
        errors.append("Password must contain a special character or a number")
    if errors:
        return errors
    return "The password is valid"
```

selected_test.py

```
from implementation_manual import IsValidPassword
import unittest

class TestIsValidPassword(unittest.TestCase):

    def test_missing_password(self):
        result = IsValidPassword("")
        self.assertEqual(result, ["Please fill the password"])

    def test_password_none(self):
        result = IsValidPassword(None)
        self.assertEqual(result, ["Please fill the password"])

    def test_password_too_short(self):
        result = IsValidPassword("short")
        self.assertEqual(result, ["Length must be between 12 and 72 characters"])

    def test_password_too_long(self):
        result = IsValidPassword("a" * 73)
        self.assertEqual(result, ["Length must be between 12 and 72 characters"])

    def test_password_exact_min_length(self):
        result = IsValidPassword("Aa1!aaaaaaaaa")
        self.assertEqual(result, "The password is valid")

    def test_password_exact_max_length(self):
```

```
        result =
IsValidPassword("Aa1!aaaa
```

Test suite: openai → Implementation: amazon

implementation_manual.py

```
def is_valid_password(password):
    if password is None or password == "":
        return ["Please fill the password"]
    if len(password) < 12 or len(password) > 72:
        return ["Length must be between 12 and 72 characters"]
    errors = []
    has_lower = any(c.islower() for c in password)
    has_upper = any(c.isupper() for c in password)
    has_digit_or_special = any(c.isdigit() or not c.isalnum() for c in password)
    if not has_lower:
        errors.append("Password must contain a lowercase letter")
    if not has_upper:
        errors.append("Password must contain an uppercase letter")
    if not has_digit_or_special:
        errors.append("Password must contain a special character or a number")
    if errors:
        return errors
    return "The password is valid"
```

selected_test.py

```
import unittest
from implementation_manual import IsValidPassword

class TestIsValidPassword(unittest.TestCase):

    def test_missing_password(self):
        # Arrange
        password = ""
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, ["Please fill the password"])

    def test_password_too_short(self):
        # Arrange
        password = "Ab1!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Length must be between 12 and 72 characters", result)

    def test_password_too_long(self):
        # Arrange
        password = "A" * 73 + "b1!"
        # Act
        result = IsValidPassword(password)
        # Assert
```

```
        self.assertIn("Length must be between 12 and 72 characters", result)

    def test_password_missing_lowercase(self):
        # Arrange
        password = "ABCDEFGHIJKL1!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Password must contain a lowercase letter", result)

    def test_password_missing_uppercase(self):
        # Arrange
        password = "abcdefghijk1!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Password must contain an uppercase letter", result)

    def test_password_missing_special_or_number(self):
        # Arrange
        password = "Abcdefghijkl"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Password must contain a special character or a number", result)

    def test_password_multiple_errors(self):
        # Arrange
        password = "abc"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Length must be between 12 and 72 characters", result)
        self.assertIn("Password must contain an uppercase letter", result)
        self.assertIn("Password must contain a special character or a number", result)

    def test_valid_password(self):
        # Arrange
        password = "Abcdefghijk1!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, "The password is valid")

    def test_password_exact_min_length(self):
        # Arrange
        password = "A1bcdefghijk"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, "The password is valid")

    def test_password_exact_max_length(self):
```

```
        # Arrange
        password = "A1" + "b" * 69 + "!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, "The password is valid")

    def test_password_with_only_required_characters(self):
        # Arrange
        password = "aA1" + "b" * 9
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, "The password is valid")

if __name__ == "__main__":
    unittest.main()
```

Test suite: openai → Implementation: deepseek

implementation_manual.py

```
def IsValidPassword(password):
    errors = []
    if not password:
        errors.append("Please fill the password")
        return errors
    if len(password) < 12 or len(password) > 72:
        errors.append("Length must be between 12 and 72 characters")
    if not any(c.islower() for c in password):
        errors.append("Password must contain a lowercase letter")
    if not any(c.isupper() for c in password):
        errors.append("Password must contain an uppercase letter")
    if not any(c.isdigit() or not c.isalnum() for c in password):
        errors.append("Password must contain a special character or a number")
    if not errors:
        return "The password is valid"
    return errors
```

selected_test.py

```
import unittest
from implementation_manual import IsValidPassword

class TestIsValidPassword(unittest.TestCase):

    def test_missing_password(self):
        # Arrange
        password = ""
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, ["Please fill the password"])

    def test_password_too_short(self):
        # Arrange
        password = "Ab1!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Length must be between 12 and 72 characters", result)

    def test_password_too_long(self):
        # Arrange
        password = "A" * 73 + "b1!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Length must be between 12 and 72 characters", result)

    def test_password_missing_lowercase(self):
```

```
        # Arrange
        password = "ABCDEFGHIJKL1!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Password must contain a lowercase letter", result)

    def test_password_missing_uppercase(self):
        # Arrange
        password = "abcdefghijk1!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Password must contain an uppercase letter", result)

    def test_password_missing_special_or_number(self):
        # Arrange
        password = "Abcdefghijkl"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Password must contain a special character or a number", result)

    def test_password_multiple_errors(self):
        # Arrange
        password = "abc"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Length must be between 12 and 72 characters", result)
        self.assertIn("Password must contain an uppercase letter", result)
        self.assertIn("Password must contain a special character or a number", result)

    def test_valid_password(self):
        # Arrange
        password = "Abcdefghijk1!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, "The password is valid")

    def test_password_exact_min_length(self):
        # Arrange
        password = "A1bcdefghijk"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, "The password is valid")

    def test_password_exact_max_length(self):
        # Arrange
        password = "A1" + "b" * 69 + "!"
        # Act
```

```
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, "The password is valid")

    def test_password_with_only_required_characters(self):
        # Arrange
        password = "aA1" + "b" * 9
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, "The password is valid")

if __name__ == "__main__":
    unittest.main()
```

Test suite: openai → Implementation: gemma

implementation_manual.py

```
import re

def IsValidPassword(password):
    if password is None or password == "":
        return ["Please fill the password"]

    errors = []

    if not (12 <= len(password) <= 72):
        errors.append("Length must be between 12 and 72 characters")

    if not re.search(r'[a-z]', password):
        errors.append("Password must contain a lowercase letter")

    if not re.search(r'[A-Z]', password):
        errors.append("Password must contain an uppercase letter")

    if not re.search(r'[!?@#$%^&*()_0-9]', password):
        # Note: The prompt specifies [!? @#$ %^&* _ 0 -9]
        # We use a regex that covers digits and the specified special characters
        if not re.search(r'[0-9!@#$%^&*_\s]', password.replace(" ", "")):
            # Re-evaluating based on the specific prompt requirement:
            # "one digit or punctuation character ([!? @#$ %^&* _ 0 -9])"
            pass

        # Let's use a more direct approach for the specific requirement
        special_or_digit_pattern = r'[a-zA-Z0-9]' # placeholder logic
        # The prompt asks for: [!? @#$ %^&* _ 0 -9]
        # This includes digits 0-9 and specific symbols.
        if not re.search(r'[0-9!@#$%^&*_\s]', password):
             # The prompt's regex is slightly ambiguous with spaces,
             # but usually implies the set of characters.
             pass

    # Refined logic to match the exact error message requirements
    errors = []

    if password is None or password == "":
        return ["Please fill the password"]

    if not (12 <= len(password) <= 72):
        errors.append("Length must be between 12 and 72 characters")

    if not any(c.islower() for c in password):
        errors.append("Password must contain a lowercase letter")

    if not any(c.isupper() for c in password):
        errors.append("Password must contain an uppercase letter")
```

```
    # Check for digit or special character [!? @#$ %^&* _ 0 -9]
    # We check if any character in the password is a digit or in the special set
    special_chars = "!@#$%^&*_"
    has_digit_or_special = any(c.isdigit() or c in special_chars for c in password)

    if not has_digit_or_special:
        errors.append("Password must contain a special character or a number")

    if errors:
        return errors

    return "The password is valid"
```

selected_test.py

```
import unittest
from implementation_manual import IsValidPassword

class TestIsValidPassword(unittest.TestCase):

    def test_missing_password(self):
        # Arrange
        password = ""
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, ["Please fill the password"])

    def test_password_too_short(self):
        # Arrange
        password = "Ab1!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Length must be between 12 and 72 characters", result)

    def test_password_too_long(self):
        # Arrange
        password = "A" * 73 + "b1!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Length must be between 12 and 72 characters", result)

    def test_password_missing_lowercase(self):
        # Arrange
        password = "ABCDEFGHIJKL1!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Password must contain a lowercase letter", result)

    def test_password_missing_uppercase(self):
```

```
        # Arrange
        password = "abcdefghijk1!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Password must contain an uppercase letter", result)

    def test_password_missing_special_or_number(self):
        # Arrange
        password = "Abcdefghijkl"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Password must contain a special character or a number", result)

    def test_password_multiple_errors(self):
        # Arrange
        password = "abc"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Length must be between 12 and 72 characters", result)
        self.assertIn("Password must contain an uppercase letter", result)
        self.assertIn("Password must contain a special character or a number", result)

    def test_valid_password(self):
        # Arrange
        password = "Abcdefghijk1!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, "The password is valid")

    def test_password_exact_min_length(self):
        # Arrange
        password = "A1bcdefghijk"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, "The password is valid")

    def test_password_exact_max_length(self):
        # Arrange
        password = "A1" + "b" * 69 + "!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, "The password is valid")

    def test_password_with_only_required_characters(self):
        # Arrange
        password = "aA1" + "b" * 9
        # Act
```

```
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, "The password is valid")

if __name__ == "__main__":
    unittest.main()
```

Test suite: openai → Implementation: manual

implementation_manual.py

```
def IsValidPassword(password):
    errors=[]
    if len(password)==0:
        return "Please fill the password"
    if len(password)<12 or len(password)>72:
        errors.append("Length must be between 12 and 72 characters")
    lowerCase=False
    upperCase=False
    num=False
    special=False
    for char in password:
        if(char.isdigit()):
            num=True
        if(char.islower()):
            lowerCase=True
        if(char.isupper()):
            upperCase=True
        if(not char.isalnum()):
            special=True
    if not lowerCase:
        errors.append("Password must contain a lowercase letter")
    if not upperCase:
        errors.append("Password must contain an uppercase letter")
    if not special or not num:
        errors.append("Password must contain a special character or a number")
    if len(errors)==0:
        return "The password is valid"
    else:
        return errors
```

selected_test.py

```
import unittest
from implementation_manual import IsValidPassword

class TestIsValidPassword(unittest.TestCase):

    def test_missing_password(self):
        # Arrange
        password = ""
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, ["Please fill the password"])

    def test_password_too_short(self):
        # Arrange
```

```
        password = "Ab1!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Length must be between 12 and 72 characters", result)

    def test_password_too_long(self):
        # Arrange
        password = "A" * 73 + "b1!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Length must be between 12 and 72 characters", result)

    def test_password_missing_lowercase(self):
        # Arrange
        password = "ABCDEFGHIJKL1!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Password must contain a lowercase letter", result)

    def test_password_missing_uppercase(self):
        # Arrange
        password = "abcdefghijk1!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Password must contain an uppercase letter", result)

    def test_password_missing_special_or_number(self):
        # Arrange
        password = "Abcdefghijkl"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Password must contain a special character or a number", result)

    def test_password_multiple_errors(self):
        # Arrange
        password = "abc"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Length must be between 12 and 72 characters", result)
        self.assertIn("Password must contain an uppercase letter", result)
        self.assertIn("Password must contain a special character or a number", result)

    def test_valid_password(self):
        # Arrange
        password = "Abcdefghijk1!"
        # Act
        result = IsValidPassword(password)
```

```
        # Assert
        self.assertEqual(result, "The password is valid")

    def test_password_exact_min_length(self):
        # Arrange
        password = "A1bcdefghijk"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, "The password is valid")

    def test_password_exact_max_length(self):
        # Arrange
        password = "A1" + "b" * 69 + "!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, "The password is valid")

    def test_password_with_only_required_characters(self):
        # Arrange
        password = "aA1" + "b" * 9
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, "The password is valid")

if __name__ == "__main__":
    unittest.main()
```

Test suite: openai → Implementation: mistral

implementation_manual.py

```
def IsValidPassword(password):
    errors = []
    if not password:
        errors.append("Please fill the password")
        return errors

    if len(password) < 12 or len(password) > 72:
        errors.append("Length must be between 12 and 72 characters")

    has_lower = any(c.islower() for c in password)
    if not has_lower:
        errors.append("Password must contain a lowercase letter")

    has_upper = any(c.isupper() for c in password)
    if not has_upper:
        errors.append("Password must contain an uppercase letter")

    has_special_or_digit = any(c.isdigit() or c in "!?@#$%^&*_-" for c in password)
    if not has_special_or_digit:
        errors.append("Password must contain a special character or a number")

    if not errors:
        return "The password is valid"
    return errors
```

selected_test.py

```
import unittest
from implementation_manual import IsValidPassword

class TestIsValidPassword(unittest.TestCase):

    def test_missing_password(self):
        # Arrange
        password = ""
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, ["Please fill the password"])

    def test_password_too_short(self):
        # Arrange
        password = "Ab1!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Length must be between 12 and 72 characters", result)

    def test_password_too_long(self):
```

```
        # Arrange
        password = "A" * 73 + "b1!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Length must be between 12 and 72 characters", result)

    def test_password_missing_lowercase(self):
        # Arrange
        password = "ABCDEFGHIJKL1!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Password must contain a lowercase letter", result)

    def test_password_missing_uppercase(self):
        # Arrange
        password = "abcdefghijk1!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Password must contain an uppercase letter", result)

    def test_password_missing_special_or_number(self):
        # Arrange
        password = "Abcdefghijkl"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Password must contain a special character or a number", result)

    def test_password_multiple_errors(self):
        # Arrange
        password = "abc"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Length must be between 12 and 72 characters", result)
        self.assertIn("Password must contain an uppercase letter", result)
        self.assertIn("Password must contain a special character or a number", result)

    def test_valid_password(self):
        # Arrange
        password = "Abcdefghijk1!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, "The password is valid")

    def test_password_exact_min_length(self):
        # Arrange
        password = "A1bcdefghijk"
        # Act
```

```
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, "The password is valid")

    def test_password_exact_max_length(self):
        # Arrange
        password = "A1" + "b" * 69 + "!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, "The password is valid")

    def test_password_with_only_required_characters(self):
        # Arrange
        password = "aA1" + "b" * 9
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, "The password is valid")

if __name__ == "__main__":
    unittest.main()
```

Test suite: openai → Implementation: openai

implementation_manual.py

```
def IsValidPassword(password):
    errors = []
    if not password:
        errors.append("Please fill the password")
        return errors
    if not isinstance(password, str):
        errors.append("Please fill the password")
        return errors
    length = len(password)
    if length < 12 or length > 72:
        errors.append("Length must be between 12 and 72 characters")
    if not any(c.islower() for c in password):
        errors.append("Password must contain a lowercase letter")
    if not any(c.isupper() for c in password):
        errors.append("Password must contain an uppercase letter")
    if not any(c.isdigit() or c in "!@#$%^&*_?" for c in password):
        errors.append("Password must contain a special character or a number")
    if errors:
        return errors
    return "The password is valid"
```

selected_test.py

```
import unittest
from implementation_manual import IsValidPassword

class TestIsValidPassword(unittest.TestCase):

    def test_missing_password(self):
        # Arrange
        password = ""
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, ["Please fill the password"])

    def test_password_too_short(self):
        # Arrange
        password = "Ab1!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Length must be between 12 and 72 characters", result)

    def test_password_too_long(self):
        # Arrange
        password = "A" * 73 + "b1!"
        # Act
        result = IsValidPassword(password)
```

```
        # Assert
        self.assertIn("Length must be between 12 and 72 characters", result)

    def test_password_missing_lowercase(self):
        # Arrange
        password = "ABCDEFGHIJKL1!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Password must contain a lowercase letter", result)

    def test_password_missing_uppercase(self):
        # Arrange
        password = "abcdefghijk1!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Password must contain an uppercase letter", result)

    def test_password_missing_special_or_number(self):
        # Arrange
        password = "Abcdefghijkl"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Password must contain a special character or a number", result)

    def test_password_multiple_errors(self):
        # Arrange
        password = "abc"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertIn("Length must be between 12 and 72 characters", result)
        self.assertIn("Password must contain an uppercase letter", result)
        self.assertIn("Password must contain a special character or a number", result)

    def test_valid_password(self):
        # Arrange
        password = "Abcdefghijk1!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, "The password is valid")

    def test_password_exact_min_length(self):
        # Arrange
        password = "A1bcdefghijk"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, "The password is valid")
```

```
    def test_password_exact_max_length(self):
        # Arrange
        password = "A1" + "b" * 69 + "!"
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, "The password is valid")

    def test_password_with_only_required_characters(self):
        # Arrange
        password = "aA1" + "b" * 9
        # Act
        result = IsValidPassword(password)
        # Assert
        self.assertEqual(result, "The password is valid")

if __name__ == "__main__":
    unittest.main()
```